ОПИСАНИЕ
протокола CANopen в реализации ООО «НПФ ВЕКТОР»

(по заказу АО «НИИЭТ»)

Москва 25.04.2017

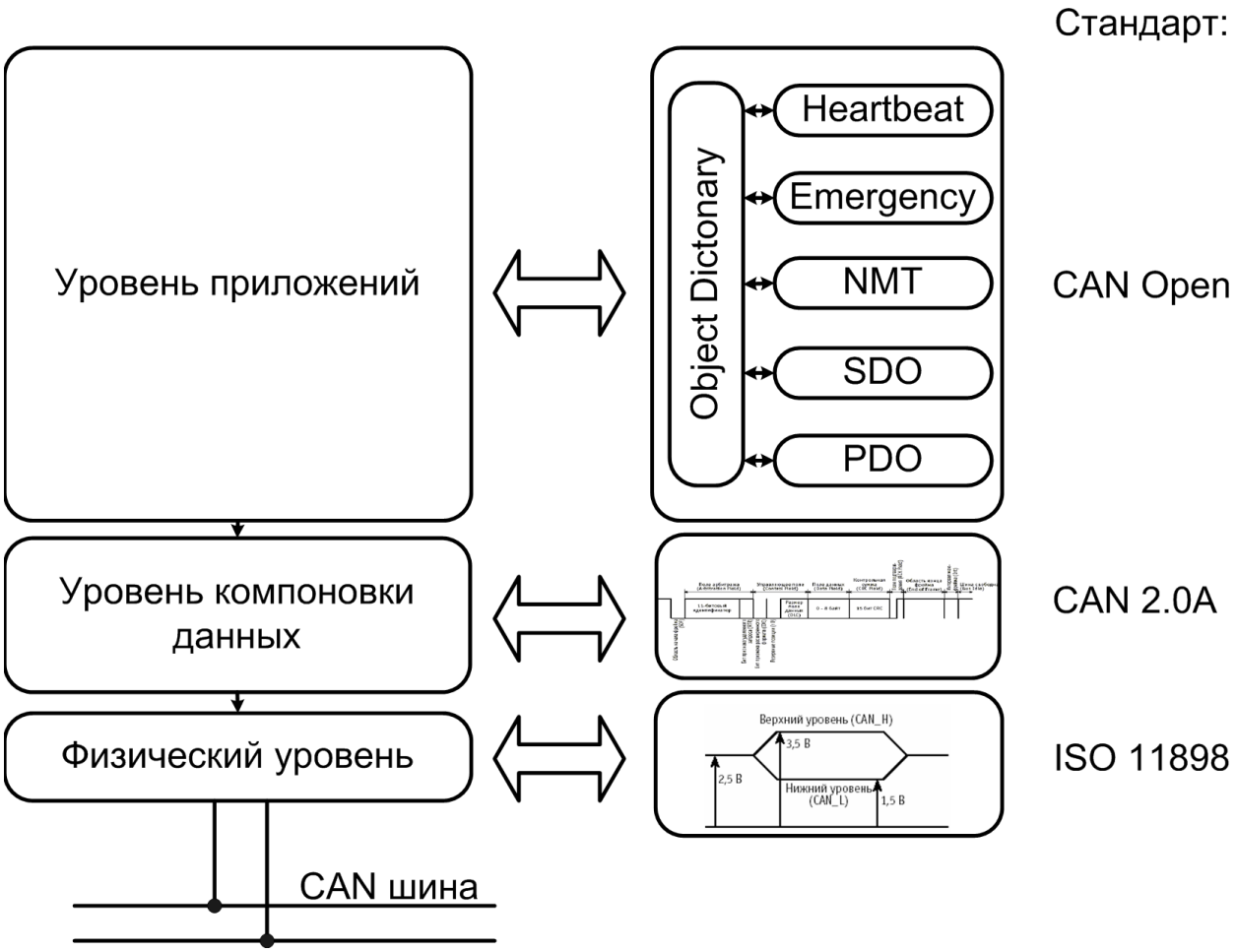
Оглавление

1. Общие сведения ... 3
2. Словарь объектов CANopen ... 6
2.1. Структура словаря ... 6
2.2. Объект словаря ... 8
3. Сервисы CANopen ... 10
3.1. Heartbeat-сервис ... 10
3.2. NMT-сервис ... 13
3.3. SDO-сервис ... 13
3.3.1. Реализация серверной части SDO-сервиса ... 13
3.3.1.1. Запись параметров в сервер ... 14
3.3.1.2. Чтение параметров сервера ... 14
3.3.1.3. Обработка ошибок SDO ... 15
3.3.1. Реализация клиентской части SDO-сервиса ... 16
3.4. PDO-сервис ... 18
3.4.1. Настройка RPDO ... 19
3.4.1.1. Коммуникационная настройка RPDOx (x = 1…8) ... 19
3.4.1.2. Настройка отображения RPDOx (x = 1…8) ... 19
3.4.1.3. Настройка вызова callback функций ... 20
3.4.1.4. Режим мульти PDO ... 20
3.4.2. Настройка TPDO ... 21
3.4.2.1. Коммуникационная настройка TPDOx (x = 1…8) ... 21
3.4.2.2. Настройка отображения TPDOx (x = 1…8) ... 22
4. Реализация драйвера CANopen ... 25
4.1. Состав драйвера ... 25
4.2. Конфигурация ... 25
4.3. Инициализация ... 28
4.4. Тактирование процессов ... 29
4.5. Функции работы с параметрами ... 30
4.5.1. Использование ЭнОЗУ (пользовательской памяти) ... 30
4.5.2. Варианты распределения пользовательской памяти при использовании двух модулей CAN ... 31
4.5.3. Функции сохранения и загрузки параметров ... 33
4.6. Интерфейс блочной передачи ... 34
4.7. Взаимодействие с драйвером CANopen через другие интерфейсы связи ... 37
5. Минимально необходимая конфигурация профиля CANopen ... 39

1. Общие сведения

Драйвер CANopen (стек протоколов) — это программа для микроконтроллера, реализующая протокол CANopen на основе интерфейса связи CAN (см. CiA DS301 V4.0.2 – CANopen application layer and communication profile).

Структура организации протокола представлена на рисунке ниже:



Драйвер реализует в себе набор сервисов (см. главу 3: «Сервисы CANopen») для управления узлами сети, контроля целостности сети, обмена оперативной и сервисной информацией и др. Взаимодействие сервисов драйвера с пользовательскими переменными осуществляется через так называемый словарь объектов. Словарь объектов определяет формат представления пользовательской переменной через драйвер CANopen, текст, значение по умолчанию, минимум, максимум и другую информацию (см. главу 2: «Словарь объектов CANopen»).

Помимо стандартных сервисов, данная реализация драйвера имеет расширения по сравнению со стандартным CANopen, однако они организованы в рамках протокола CANopen и являются «надстройками», не нарушая стандарт.

Драйвер поддерживает следующие стандартные CANopen сервисы:

- Сервис NMT (network management) - сервис служит для управления состояниями узлов сети со стороны мастера сети. Сервис позволяет изменять состояния любых узлов, находящихся в сети. Подробнее см. п. 3.2
- Сервис Heartbeat. Сервис, позволяющий узлам контролировать наличие других узлов в сети и автоматически изменять собственное состояние при изменении состава сети. Подробнее см. п. 3.1.
- Expedited SDO сервис. Подтверждаемый сервис передачи данных между клиентом и сервером. Сервером является устройство, к объектному словарю которого

пытается получить доступ клиент. То есть как сервером, так и клиентом может быть любое устройство в сети. Данный сервис позволяет читать и записывать параметры сервера по запросу клиента (expedited – передаваемая информация умещается в одной посылке (до 32 бит полезных данных), передача блоков данных, структур и т.д. не поддерживается). Данный сервис обычно используется для настройки, конфигурирования и взаимодействия узлов сети с пультом оперативного управления, программой UniCON и др. Иными словами, данный сервис обычно используется для передачи информации не в режиме реального времени. Подробнее см. п. 3.3.

- PDO сервис. Неподтверждаемый сервис передачи оперативной информации. Драйвер поддерживает 8 свободно конфигурируемых PDO на прием (RPDO) и 8 свободно конфигурируемых PDO на передачу (TPDO). Это основные сообщения обмена между устройствами внутри сети в режиме реального времени и обеспечивающие поддержание технологического процесса. Мастер конфигурирует в каждом устройстве, какие PDO и с какими данными и по какому событию устройство будет принимать, какие отправлять. Одно сообщение PDO содержит до 64 бит данных, которые набираются из разных объектов словаря свободно. Подробнее см. п. 3.4.

Драйвер, реализованный ООО «НПФ ВЕКТОР» по заказу АО «НИИЭТ», имеет следующие расширенные функции, поддерживаемые ПО верхнего уровня UniCON:

- Режим «MultiPDO». Режим расширяет возможности сервиса PDO по приему RPDO – включение данного режима позволяет узлу принимать вместо 8 RPDO до 8*127 RPDO. Данный режим полезен при реализации многоконтроллерной системы управления, когда 8 RPDO становится недостаточно.
- Возможность сохранять/загружать значения параметров словаря объектов в энергонезависимую память устройства (eeprom, flash и т.п.). Эта функция имеется в большинстве других реализаций стека CANopen.
- Предоставление доступа к структуре словаря объектов для возможности «скачивания» структуры по сети CANopen. Т.е. устройство верхнего уровня (мастер) может, не зная состава словаря объектов конкретного устройства, загрузить его непосредственно из устройства для дальнейшего взаимодействия. Функция реализована посредством обмена данными через некоторый набор «стандартных» объектов словаря, которые есть во всех устройствах CANopen с драйвером реализации ООО «НПФ ВЕКТОР». В сторонних реализациях CANopen для работы с устройством предполагается обязательное наличие так называемого EDS файла, описывающего словарь объектов. Другими словами, содержимое этого файла в реализации драйвера ООО «НПФ ВЕКТОР» может скачиваться по сети.
- Расширенные типы данных, поддерживающие так называемые масштабирующие коэффициенты и единицы измерений. Т.е. для параметра можно указать не только что он, например, знаковый длиной 32 разряда, но и что цена младшего разряда составляет 0.00125 Ампер. Вместе с функцией загрузки словаря по сети это позволяет отображать пользователю параметры не просто в виде «сухих» чисел, а непосредственно в вольтах, амперах и т.п. Программисту же это позволяет не пытаться внутри программы использовать физические величины (чтобы число 125, скажем, обозначало 12,5 А), а реализовывать систему управления в независимых относительных единицах измерения. Это делает систему управления универсальной с точки зрения масштабируемости — программа будет одинаково хорошо работать как для преобразователя частоты мощностью 1кВт, так и для преобразователя частоты мощностью 1000кВт.
- Поддержка функций обратного вызова при доступе к параметру. Программист устройства может сконфигурировать драйвер CANopen так, чтобы при обращении

к определенному объекту словаря драйвер вызвал произвольную пользовательскую функцию. Это удобно для различных надстроек на основе словаря CANopen: например, для выгрузки значений журнала аварий устройства и т.п. Программист может сразу же после задания средой верхнего уровня в одном объекте, например, номера ячейки журнала аварий в другой объект словаря выставить данные с кодом аварии.

- Функция цифрового осциллографирования внутри микроконтроллера. Для отладки сложных систем управления требуется видеть осциллограммы процессов внутри системы управления: изменение тех или иных переменных во времени. Драйвер CANopen, разработанный ООО «НПФ ВЕКТОР» по заказу АО «НИИЭТ» имеет интерфейс для модуля цифрового осциллографа и позволяет пользователю выбрать из словаря объектов любые четыре параметра, после чего передает их адреса переменных внутри микроконтроллера модулю осциллографирования (который записывает в массивы, как изменяются эти значения во времени). После записи осциллограмм при помощи функции обратного вызова программа верхнего уровня UniCON загружает и отображает эти осциллограммы. Также возможна запись осциллограмм по триггеру. Для программиста системы управления доступно 32 независимых триггера, которые могут запускать процесс осциллографирования по конкретным событиям в системе управления. У пользователя программы верхнего уровня UniCON есть возможность указать по какому событию он хочет записать в данный момент осциллограмму.
- Функция блочной передачи данных Блочная передача реализует возможность потокового приема и передачи данных. Максимальный размер блока 256 16-и разрядных слов (512 байт). Блочная передача оказывается полезной в случаях когда необходимо передать большой объем низкоприоритетной информации, например, приложению UniCON при загрузке осциллограмм (на вкладке «Выборка») из устройства. Подробнее см. п. 4.6.

2. Словарь объектов CANopen

Основной функциональной единицей протокола CANopen является объект. Под объектом может пониматься набор данных, несущих информацию о параметрах (например, показания датчика температуры), конфигурации узла или сети, возникших ошибках и т.п. Поэтому для устройства (узла) необходимым условием работы в сети является наличие словаря, представляющего собой группу доступных в определенном порядке объектов.

2.1. Структура словаря

Реализованный словарь объектов CANopen фактически описывает параметры устройства – узла сети, обеспечивая доступ к его переменным в ОЗУ контроллера. Согласно стандарту CANopen, адресное пространство (индексы) разделено на несколько диапазонов:

Адрес (индекс)	Объекты
0001h – 001Fh	Стандартные типы данных
0020h – 003Fh	Комплексные типы данных
0040h – 005Fh	Комплексные типы данных производителя
0060h – 007Fh	Типы специфичные для профиля оборудования (простые)
0080h – 009Fh	Типы специфичные для профиля оборудования (комплексные)
00A0h – 0FFFh	Зарезервировано
1000h – 1FFFh	Область коммуникационного профиля
2000h – 5FFFh	Область профиля производителя
6000h – 9FFFh	Область профиля стандартных устройств (стандартизованных)
A000h – BFFFh	Стандартизованная интерфейсная область
C000h – FFFFh	Зарезервировано

Подмножество протокола CANopen, реализованное ООО «НПФ ВЕКТОР» по заказу АО «НИИЭТ», использует только диапазоны 1000 – 1FFF («Область коммуникационного профиля») и 2000 – 5FFF («Область профиля производителя»).

В коммуникационном профиле описываются сетевые настройки. Этот профиль стандартный и по большей части соответствует спецификации DS-301 v.4.02:

Адрес	Описание
1000h – 11FFh	Общие коммуникационные параметры
1200h – 13FFh	Конфигурация SDO
1400h – 15FFh	Конфигурация RPDO
1600h – 17FFh	Распределение RPDO
1800h – 19FFh	Конфигурация TPDO
1A00h – 1BFFh	Распределение TPDO

Профиль производителя включает:

Адрес	Описание
2000h – 207Fh	Дополнительные параметры связи, отсутствующие в стандартном коммуникационном профиле
2080h – 20FFh	Командный фрейм для функций драйвера CANopen
2100h – 211Fh	Масштабирующие коэффициенты
2400h – 2FFFh	Служебные параметры
3000h – 5FFFh	Параметры и переменные пользовательского проекта

Поддерживаются следующие форматы (типы) параметров:

U16	Беззнаковое слово. Может интерпретироваться как дробное беззнаковое слово с фиксированной точкой (IQ-формат)*.
U32	Беззнаковое двойное слово. Может интерпретироваться как дробное двойное беззнаковое слово с фиксированной точкой (IQ-формат)*.
I16	Знаковое слово. Может интерпретироваться как дробное знаковое слово с фиксированной точкой (IQ-формат)*.
I32	Знаковое двойное слово. Может интерпретироваться как дробное двойное знаковое слово с фиксированной точкой (IQ-формат)*.
Битовое поле	Последовательность битов длиной от 1 (boolean) до 8 бит.
Float32	Вещественное число.

* Примечание: интерпретация целочисленных значений как дробных с фиксированной точкой (IQ-формат) является расширением стандарта CANopen в реализации ООО «НПФ ВЕКТОР».

По типу доступа параметры делятся на:

Наблюдаемый	Доступен только для чтения.
Редактируемый	Доступен для чтения и для записи.
Редактируемый защищённый	Доступен для чтения. Доступен для записи только в том случае, если переменная «co_protectBit»* (адрес в словаре – 2005.00) равна нулю. Это может быть полезно, например, для защиты некоторых параметров от изменения при работающем электроприводе (управление этой переменной – задача пользовательской программы).
Секретный редактируемый защищённый	Доступен для чтения. Доступен для записи только в том случае, если нулю равна не только переменная «co_protectBit», но и переменная «co_secretProtectBit» – дополнительный уровень защиты доступа (управление этими переменными – задача пользовательской программы).

Примечание*:

Полное имя переменной зависит от того, с каким модулем CAN мы работаем. Для модуля CAN1 полное имя будет «co1_vars.co_protectBit», для модуля CAN2 – «co2_vars.co_protectBit».

2.2. Объект словаря

С точки зрения адресации словарь объектов имеет двухуровневую структуру, т.е. адрес объекта состоит из двух компонентов: индекс и подындекс. В настоящем Описании далее будет принято отображение адреса в виде шестнадцатеричных чисел: XXXX.YY (XXXX – индекс, YY – подындекс).

Существует два типа объектов словаря:

- Объект–параметр устройства, например:

№	Индекс	Поды	Длинный текст	Короткий текст	Английский текст	Формат	Адрес
478	3000h	00h	Статус преобразователя	Статус	Статус преобразователя	R.Бит.текст. 0-5	drv_status.all

Такой объект может иметь адрес вида XXXXh.00h (одиночный параметр), либо входить в состав так называемого корневого объекта:

- Корневой объект–контейнер: содержит внутри себя несколько (до 255) объектов-параметров, объединённых по какому-либо признаку. Такой корневой объект имеет адрес вида XXXXh.00h, а содержащиеся в нём объекты-параметры будут иметь адреса вида XXXXh.01h, XXXXh.02h и т.д. Например, параметры задатчика интенсивности объединены под индексом 5000h:

№	Индекс	Поды	Длинный текст	Короткий текст	Английский текст	Формат	Адрес
485	5000h	00h	Задатчик интенсивности	ЗИ	Ramp	Корневой	
486	5000h	01h	T разгона до номинала	rmpT	T разгона до номинала	P.Q-тип.I32 8,24 м.к.№09h	rmp.T
487	5000h	02h	Вход ЗИ	Вход.ЗИ	Ramp input	R.Q-тип.I32 8,24 м.к.№14h	rmp.input
488	5000h	03h	Выход ЗИ	Выход.ЗИ	Ramp output	R.Q-тип.I32 8,24 м.к.№14h	rmp.output

Объект имеет следующие атрибуты*:

Поле	Описание
Индекс	Старшая часть адреса.
Подындекс	Младшая часть адреса (адрес внутри индекса).
Текст	Ссылка-идентификатор на запись с текстом, который отображается в качестве названия параметра (номер текста).
Формат	В этом поле кодируется формат параметра (корневой / целый / вещественный / битовое поле / число дробных разрядов / тип доступа и т.п.), а также используемый масштабирующий коэффициент.
Адрес	Применим только к некорневым объектам-параметрам. Здесь указывается имя (адрес) переменной в ПО устройства.
Группа	Используется только для объектов с адресом вида XXXXh.00h (только для элементов верхнего уровня вложенности, т.е. «индексов»). Здесь выбирается, к какой группе относится данный объект (и его подындексы для корневого объекта). В данном поле хранится номер-смещение текста в перечислении с названиями групп параметров.
По умолчанию	Применимо только к некорневым редактируемым объектам-параметрам. Здесь задаётся значение параметра по умолчанию («заводская» уставка).
Мин./Макс.	Применимо только к некорневым редактируемым объектам-параметрам. Здесь задаётся минимально/максимально допустимое значение параметра.
CallBack	Используется только для объектов с адресом вида XXXXh.00h (только для элементов верхнего уровня вложенности, т.е. «индексов»). В данном поле указывается имя (адрес) callback-функции в ПО устройства, которая вызывается драйвером CANopen при SDO-запросе (на чтение/запись) к данному индексу или любому его подындексу, восстановлении значения по умолчанию или загрузке значения из ЭнОЗУ с помощью функций co_ODexpositor_instaurationDefault и co_ODexpositor_paramInstauration

	соответственно. Если индекс имеет несколько подындексов, то вызов callback функции будет осуществляться каждый раз при доступе к любому подындексу данного индекса. Это необходимо учитывать при организации структуры словаря и ассоциации индексам callback функций.

* Примечание: атрибуты объектов словаря задаются пользователем с помощью приложения COODEdit.

3. Сервисы CANopen

Работа протокола CANopen реализована на базе так называемых сервисов (служб).

3.1. Heartbeat-сервис

Сервис реализует контрольное тактирование в виде периодической передачи специального широковещательного heartbeat-сообщения (heartbeat – «сердцебиение»), обеспечивая слежение за состоянием устройств в сети. Каждое устройство в сети генерирует heartbeat-сообщение с определенной частотой (являясь «Heartbeat Producer») и одновременно «слушает» сообщения других (являясь «Heartbeat Consumer»).

Периодичность посылки heartbeat-сообщения определяется переменной «co_PHBT» (параметр 1017.00, по умолчанию 1000 мс).

Сообщение отсылается короткой однобайтовой посылкой:

Поле	ID (идентификатор сообщения)	DLC (число байт данных)	Данные: D0
Значение	700h + № узла	1	N

Где «N» – код сетевого состояния устройства:

Состояние	Описание
0 – BOOTUP	Сообщение с данным кодом посылается всего один раз, после начальной инициализации устройства, после чего оно переходит в состояние «PREOPERATIONAL».
4 – STOPPED	Функционируют только сервисы Heartbeat и NMT.
5 – OPERATIONAL	Номинальный режим работы. Функционируют все сервисы: Heartbeat, NMT, SDO, PDO.
127 – PREOPERATIONAL	В этом состоянии возможна настройка (модификация параметров) устройства извне с помощью SDO-сервиса. В данном состоянии функционируют сервисы: Heartbeat, NMT, SDO.

Например, heartbeat-сообщение от узла №50, находящегося в состоянии «OPERATIONAL», будет иметь вид:

ID	DLC	D0
00000732h	1	05h

Текущее состояние устройства доступно для просмотра через переменную «co_nodeState» (параметр 2015.00).

Каждое устройство имеет битовые heartbeat-флаги, сигнализирующие о состоянии других узлов в сети (с номерами от 1 до 127):

Адрес	Имя переменной	Описание
2010.01	co_heartbeatFlag0	Heartbeat-флаги узлов 0-31 (нулевой бит всегда = 0, т.к. узла с номером 0 в сети быть не может)
2010.02	co_heartbeatFlag1	Heartbeat-флаги узлов 32-63
2010.03	co_heartbeatFlag2	Heartbeat-флаги узлов 64-95
2010.04	co_heartbeatFlag3	Heartbeat-флаги узлов 96-127

Эти флаги в узле-приёмнике устанавливаются по мере прихода heartbeat-сообщений от других устройств в зависимости от их сетевого состояния по следующей схеме:

Отправитель / Приемник	BOOTUP	STOPPED	PREOPERATIONAL	OPERATIONAL
STOPPED	1	1	1	1
PREOPERATIONAL	0	0	1	1
OPERATIONAL	0	0	0	1

Флаг, соответствующий своему собственному номеру узла, всегда выставляется равным 1.

Каждое устройство также имеет редактируемые битовые heartbeat-маски, определяющие минимально необходимый набор устройств в сети для того, чтобы оставаться в состоянии OPERATIONAL:

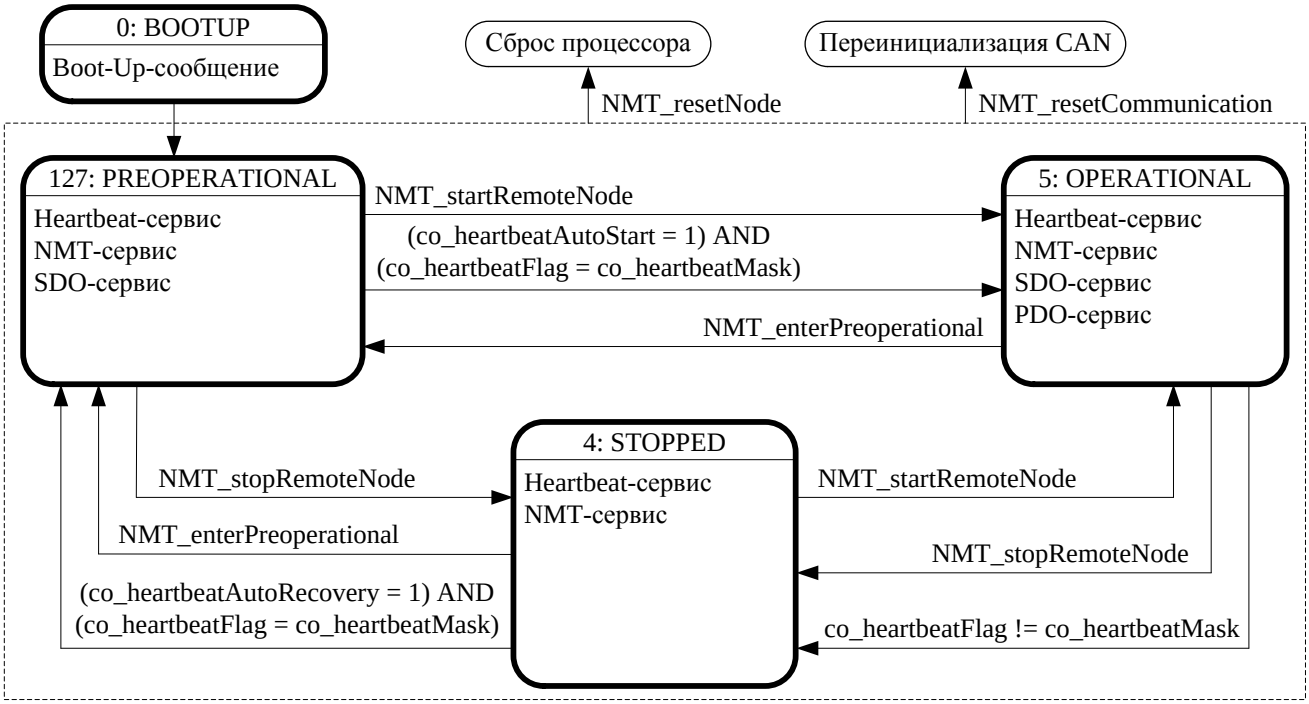
Адрес	Имя переменной	Описание
2011.01	co_heartbeatMask0	Heartbeat-маска узлов 0-31 (нулевой бит должен быть = 0, т.к. узла с номером 0 в сети быть не может)
2011.02	co_heartbeatMask1	Heartbeat-маска узлов 32-63
2011.03	co_heartbeatMask2	Heartbeat-маска узлов 64-95
2011.04	co_heartbeatMask3	Heartbeat-маска узлов 96-127

Время ожидания heartbeat-сообщений от других узлов определяется переменной «co_CHBT» (параметр 1016.01, по умолчанию 2000 мс). С периодичностью, задаваемой этим параметром, производится сверка соответствия масок «co_heartbeatMask#» и флагов «co_heartbeatFlag#». Если флаги покрывают всю маску, то устройство остаётся в состоянии OPERATIONAL, а флаги очищаются до следующих heartbeat-сообщений (т.е. они циклически меняют своё состояние 0↔1). Если маска оказалась не закрыта, например, какое-то устройство пропало из сети, то происходит переход в состояние STOPPED.

Переход узла из одного состояния в другое осуществляется по командам мастера CANopen-сети посредством NMT-сервиса (см. п.3.2 «NMT-сервис»). В сети без мастера возможен автоматический выход узла из состояния STOPPED, что контролируется следующими битовыми параметрами:

Адрес	Имя переменной	Описание
2014.00	co_heartbeatAutoRecovery	Если =1, то при появлении в сети всех обязательных устройств (флаги «co_heartbeatFlag#» совпадают с маской «co_heartbeatMask#») узел автоматически переходит из состояния STOPPED в состояние PREOPERATIONAL.
2012.00	co_heartbeatAutoStart	Если =1, то при появлении в сети всех обязательных устройств узел автоматически переходит из состояния PREOPERATIONAL в состояние OPERATIONAL.

Диаграмма смены сетевого состояния узла выглядит следующим образом:



Здесь «NMT_...» – это команды, получаемые через NMT-сервис от мастера CAN-сети (расшифровку команд см. в п.3.2 «NMT-сервис»).

Также в драйвере CANopen имеются дополнительные флаги наличия устройств в сети (в словаре объектов они отсутствуют), формируемые без учёта сетевого состояния узлов:

Имя переменной	Описание
co_devicePresentFlag0	Флаги наличия узлов 0-31 (нулевой бит всегда = 0, т.к. узла с номером 0 в сети быть не может)
co_devicePresentFlag1	Флаги наличия узлов 32-63
co_devicePresentFlag2	Флаги наличия узлов 64-95
co_devicePresentFlag3	Флаги наличия узлов 96-127

Эти флаги формируются в узле-приёмнике следующим образом: как только от какого-то узла сети приходит heartbeat-сообщение, выставляется соответствующий флаг «co_devicePresentFlag#». С периодичностью, определяемой параметром «co_CHBT», этот флаг обнуляется до следующего прихода heartbeat-сообщения (т.е. он циклически меняет своё состояние 0↔1).

Для более удобного мониторинга присутствия узлов сети в драйвере CANopen имеются глобальные флаги наличия устройств (в словаре объектов они отсутствуют), которые постоянно находятся в состоянии «1», пока от соответствующего устройства приходят heartbeat-сообщения (в пределах таймаута «co_CHBT»), независимо от его сетевого состояния:

Имя переменной	Описание
co_deviceGlobalPresent0	Глобальные флаги наличия узлов 0-31 (нулевой бит всегда = 0, т.к. узла с номером 0 в сети быть не может)
co_deviceGlobalPresent1	Глобальные флаги наличия узлов 32-63
co_deviceGlobalPresent2	Глобальные флаги наличия узлов 64-95
co_deviceGlobalPresent3	Глобальные флаги наличия узлов 96-127

3.2. NMT-сервис

NMT-сервис (network management) служит для управления состояниями узлов сети со стороны мастера. Сервис позволяет удалённо изменять сетевое состояние устройства. В безмастерной сети используется автозапуск. В сети с мастером автозапуск тоже возможен, хотя обычно управление ведётся через NMT.

Для смены состояния удалённого узла мастер сети может воспользоваться следующей функцией драйвера CANopen:

```
void co_sendNMT(TCo_OdVars* ppc, Uint16 nmtCommand, Uint16 nmtNODE_ID)
```

Описание:

Функция смены состояния удалённого узла.

Функция возвращает управление сразу, не дожидаясь отправки сообщения в сеть CAN.

Параметры вызова:

- ppc – указатель на структуру используемого модуля CAN драйвера CANopen.
- nmtCommand – код NMT-команды:
  1 – перевести в состояние OPERATIONAL;
  2 – перевести в состояние STOPPED;
  128 – перевести в состояние PREOPERATIONAL;
  129 – перезапустить устройство (путём программного сброса процессора);
  130 – перезапустить коммуникационный модуль CAN (включая перенастройку PDO).
- nmtNODE_ID – номер узла сети, которому посылается команда.

Внимание

Если разрешён автозапуск (см. описание Heartbeat-сервиса, параметры 2012.00 и 2014.00), то перевод узла из состояния OPERATIONAL в PREOPERATIONAL приведет к его автоматическому перезапуску снова в состояние OPERATIONAL. Аналогично, если heartbeat-флаги не покрывают heartbeat-маску, и узел находится в состоянии PREOPERATIONAL, то при попытке перевести его NMT-командой в состояние OPERATIONAL узел попадёт в режим STOPPED.

3.3. SDO-сервис

Для медленного обмена данными, например, для изменения настроек конечных устройств, просмотра банков событий и т.д. используется SDO-сервис (service data objects), работающий по принципу «клиент-сервер» («запрос-ответ»). В драйвере CANopen ООО «НПФ ВЕКТОР» реализована наиболее часто используемая ускоренная передача (expedited transfer), позволяющая читать/записывать данные длиной до 32-х разрядов. Пакетная передача (block transfer) блоков данных и структур не поддерживается.

В драйвере реализована как серверная (ответы на запросы внешних SDO-клиентов), так и клиентская (драйвер выступает в роли SDO-клиента, запрашивающего данные от внешнего SDO-сервера) части.

3.3.1. Реализация серверной части SDO-сервиса

Серверная часть SDO-сервиса предоставляет доступ (чтение/запись) внешним SDO клиентам к объектам словаря. Серверная часть SDO-сервиса работает в автоматическом режиме и не требует вмешательства пользователя.

Взаимодействие SDO-клиента и SDO-сервера осуществляется следующим образом. Протокол делится на два типа взаимодействия: это запрос параметра от сервера (SDO upload protocol) и запись параметров в сервер (SDO download protocol).

3.3.1.1. Запись параметров в сервер

Запись параметров в сервер осуществляется следующими сообщениями:

со стороны клиента формируется пакет запроса:

байты\биты	7	6	5	4	3	2	1	0
0	ccs = 001 (инициировать загрузку данных на сервер)			X = 0 (всегда ноль)	n -- количество байт данных d, не содержащих данных		e = 1 (expedited transfer)	s = 1 (размер передаваемых данных известен и указан)
1	index (индекс записываемого объекта)							
2								
3	sub-index (под-индекс записываемого объекта)							
4	байт 0 (передаваемых серверу данных)							
5	байт 1 (передаваемых серверу данных)							
6	байт 2 (передаваемых серверу данных)							
7	байт 3 (передаваемых серверу данных)							

со стороны сервера формируется ответ:

байты\биты	7	6	5	4	3	2	1	0
0	scs = 011 (ответ от сервера на инициацию загрузки данных)			X = 0 (всегда ноль)				
1	index (индекс записываемого объекта)							
2								
3	sub-index (под-индекс записываемого объекта)							
4	зарезервировано (всегда 0)							
5								
6								
7								

3.3.1.2. Чтение параметров сервера

Чтение параметров сервера осуществляется следующими сообщениями:

со стороны клиента формируется пакет запроса:

байты\биты	7	6	5	4	3	2	1	0
0	ccs = 010 (инициировать запрос данных с сервера)			X = 0 (всегда ноль)				
1	index (индекс запрашиваемого объекта)							
2								
3	sub-index (под-индекс запрашиваемого объекта)							
4	зарезервировано (всегда 0)							
5								
6								
7								

со стороны сервера формируется ответ:

байты\биты	7	6	5	4	3	2	1	0
0	scs = 010 (ответ на инициированный запрос данных с сервера)			X = 0 (всегда ноль)	n -- количество байт данных d, не содержащих данных		e = 1 (expedited transfer)	s = 1 (размер передаваемых данных известен и указан)
1	index (индекс запрошенного объекта)							
2								
3	sub-index (под-индекс запрошенного объекта)							
4	байт 0 (возвращаемых сервером данных)							
5	байт 1 (возвращаемых сервером данных)							
6	байт 2 (возвращаемых сервером данных)							
7	байт 3 (возвращаемых сервером данных)							

3.3.1.3. Обработка ошибок SDO

Сообщения об ошибках передаются в случае их возникновения в следующем формате:

байты\биты	7	6	5	4	3	2	1	0
0	cs = 4 (ответ о произошедшей ошибке)			X = 0 (всегда ноль)				
1	index (индекс запрошенного объекта)							
2								
3	sub-index (под-индекс запрошенного объекта)							
4	байт 0 (возвращаемых сервером данных)							
5	байт 1 (возвращаемых сервером данных)							
6	байт 2 (возвращаемых сервером данных)							
7	байт 3 (возвращаемых сервером данных)							

В рамках CANopen существует много кодов ошибок. В данной реализации драйвера поддерживаются следующие:

Код ошибки	Описание
0504 0001h	ccs или scs не используется или недопустим.
0601 0002h	Попытка записать в объект только для чтения
0602 0000h	Объект не существует в словаре объектов
0604 0041h	Объект не может быть размещен в PDO (причины могут быть различными: объект не существует, объект не rw при конфигурации RPDO и т.п.).
0604 0042h	Число или длина объектов превосходит размер, умещающийся в PDO.
0607 0010h	Не совпадает тип данных
0609 0011h	Под-индекс не существует
0609 0030h	Значение параметра не укладывается в диапазон допустимых значений
0800 0000h	Общая ошибка
0800 0020h	Данные не могут быть переданы или сохранены в приложении (попытка несанкционированной записи в секретный параметр)
0800 0022h	Данные не могут быть переданы или сохранены в приложении из-за текущего

	состояния устройства (попытка записи в защищенный параметр, защищенные и секретные, и простые)

3.3.1. Реализация клиентской части SDO-сервиса

Клиентская часть SDO-сервиса позволяет пользователю драйвера обращаться к объектам словаря других устройств сети (SDO-серверов), а именно – производить чтение и запись. Следует иметь ввиду, что обмен оперативной информацией, необходимой для реализации распределенной системы управления каким-либо технологическим процессом в CANopen решается с помощью PDO сервиса. SDO-сервис используется для задач конфигурации, параметрирования и т.п. Таким образом, зачастую в сети в качестве SDO-клиента выступает ПО для персонального компьютера, обеспечивающее настройку, отладку и параметрирование узлов сети – например, программа UniCON ООО «НПФ ВЕКТОР». Тем не менее, в ряде задач, например при реализации контроллера верхнего уровня, пользователю может понадобиться реализация SDO-клиента на микроконтроллере. В этом случае он может воспользоваться функциями, описанными ниже.

Для запроса значения параметра удалённого узла (сервера) клиент может воспользоваться функцией драйвера CANopen:

```
void co_SDOrequestRead(TCo_OdVars* ppc)
```

Описание:

Функция SDO-запроса значения параметра удалённого узла.

Данную функцию можно вызывать как из фоновой программы, так и из прерывания.

Функция возвращает управление сразу, не дожидаясь отправки сообщения в сеть CAN.

Параметры вызова:

- ppc – указатель на структуру используемого модуля CAN драйвера CANopen, в которой перед вызовом функции необходимо заполнить следующие поля:
  co_SDOrequestStatus – в биты 8–14 (нумерация с 0) необходимо поместить адрес узла сети, к которому выполняется запрос;
  co_SDOrequestFlag – внутренний флаг, необходимо обнулить;
  co_SDOrequestIndex – индекс запрашиваемого параметра;
  co_SDOrequestSubIndex – подындекс запрашиваемого параметра.

Пример запроса на чтение:

```
co1_vars.co_SDOrequestStatus = nodeID << 8; // nodeID - номер узла
co1_vars.co_SDOrequestFlag = 0;
co1_vars.co_SDOrequestIndex = 0x2532;  // индекс
co1_vars.co_SDOrequestSubIndex = 0x01; // подындекс
co_SDOrequestRead(&co1_vars);
```

Контроль получения ответа осуществляется по битовым полям переменной «co_SDOrequestStatus»:

15	14 8	7 0
Ready	NodeID	Error

Биты	Поле	Описание
15	Ready	Флаг готовности данных (получен ответ).
14-8	NodeID	Номер узла, к которому был направлен запрос.
7–0	Error	Код ошибки (0 – отсутствие ошибок) – см. ниже.

Если ответ на запрос получен без ошибок, запрошенные данные будут находиться в переменных «co_SDOrequestData» (младшие 16 бит) и «co_SDOrequestDataH (старшие 16 бит). Пример проверки получения ответа:

```
if ((co1_vars.co_SDOrequestStatus & 0x8000) == 0x8000) // ответ пришёл
{
    if ((co1_vars.co_SDOrequestStatus & 0x00FF) == 0x00) // ошибки нет
    {
        Uint32 data =
            ((Uint32)co1_vars.co_SDOrequestData |
            ((Uint32)co1_vars.co_SDOrequestDataH << 16));
    }
}
```

Для изменения значения параметра удалённого узла (сервера) клиент может воспользоваться функцией драйвера CANopen:

```
void co_SDOrequestWrite(TCo_OdVars* ppc)
```

Описание:

Функция SDO-запроса на запись значения параметра удалённого узла.

Данную функцию можно вызывать как из фоновой программы, так и из прерывания.

Функция возвращает управление сразу, не дожидаясь отправки сообщения в сеть CAN.

Параметры вызова:

- ppc – указатель на структуру используемого модуля CAN драйвера CANopen, в которой перед вызовом функции необходимо заполнить следующие поля:
  co_SDOrequestStatus – в биты 8–14 (нумерация с 0) необходимо поместить адрес узла сети, к которому выполняется запрос;
  co_SDOrequestFlag – внутренний флаг, необходимо обнулить;
  co_SDOrequestIndex – индекс модифицируемого параметра;
  co_SDOrequestSubIndex – подындекс модифицируемого параметра;
  co_SDOrequestData – желаемое значение параметра (младшие 16 бит);
  co_SDOrequestDataH – желаемое значение параметра (старшие 16 бит).

Пример запроса на запись:

```
co1_vars.co_SDOrequestStatus = nodeID << 8; // nodeID - номер узла
co1_vars.co_SDOrequestFlag = 0;
co1_vars.co_SDOrequestIndex = 0x5100;  // индекс
co1_vars.co_SDOrequestSubIndex = 0x01; // подындекс
Uint32 data = _IQ(0.5);     // число 0.5 в IQ-формате
co1_vars.co_SDOrequestData = (Uint16)(data);
co1_vars.co_SDOrequestDataH = (Uint16)(data >> 16);
co_SDOrequestWrite(&co1_vars);
```

Контроль выполнения запроса на запись осуществляется аналогично запросу на чтение, по битовым полям переменной «co_SDOrequestStatus».

Если запрос на запись затрагивает параметр 2000h (номер узла в сети), 2001h (скорость обмена в сети) или настройки отображения PDO, то драйвер CANopen на стороне сервера автоматически переинициализирует соответствующую коммуникационную подсистему.

Коды ошибок (биты 0-7 переменной «co_SDOrequestStatus»):

Код	Описание
0x00	Нет ошибки, запрос успешно выполнен.
0x01	Переключаемый бит не может быть изменен (попытка несанкционированного доступа к битовому параметру).
0x02	Попытка установить связь по SDO-протоколу с пакетной передачей данных (она не поддерживается драйвером CANopen).
0x03	Попытка записать в объект, предназначенный только для чтения.
0x04	Запрошенный параметр не существует в словаре объектов.
0x05	Не совпадает тип данных.
0x06	Запрошенный подындекс не существует.
0x08	Общая ошибка.

0x09	Попытка несанкционированной (флаг «co_protectBit» сброшен, но установлен флаг «co_secretProtectBit») записи в «секретный редактируемый защищённый» параметр.
0x0A	Попытка несанкционированной (установлен флаг «co_protectBit») записи в «редактируемый защищённый» или «секретный редактируемый защищённый» параметр.
0x0B	Объект не может быть размещен в PDO, т.к. он не существует.
0x0C	Число или длина объектов превосходит допустимый размер PDO.
0xFE	Неизвестная ошибка.
0xFF	Запрос был отправлен, но ответ не поступил

Драйвер CANopen никак не обрабатывает ситуацию с превышением таймаута на запрос (запрос не дошёл до сервера или ответ не дошёл до клиента) – это задача пользовательской программы.

Если для объекта (на уровне индекса) задана callback-функция, то она вызывается при внешнем SDO-запросе к данному индексу или любому его подындексу – см. п.4.2 «Конфигурация».

Эта функциональность может быть использована, например, если при изменении какого-то параметра (например, с помощью приложения UniCON) требуется совершить какие-то действия.

3.4. PDO-сервис

Оперативный (realtime) обмен данными между узлами сети, как правило, осуществляется с помощью PDO-сервиса (process data objects), обеспечивающего периодическую или событийную (по требованию пользователя) отправку запакованных широковещательных сообщений длиной до 64 бит (передача сообщений является неподтверждаемой).

PDO условно делятся на входящие (RPDO) и исходящие (TPDO). Устройства, в конкретный момент времени выдающие PDO на шину, называются производителями (producer), а принимающие эти PDO – потребителями (consumer). Драйвер CANopen, разработанный ООО «НПФ ВЕКТОР» по заказу АО «НИИЭТ», реализует асинхронный вариант PDO-обмена с возможностью принимать/отправлять до восьми конфигурируемых RPDO/TPDO для каждого узла. Количество принимаемых RPDO можно увеличить до 8*127 = 1016, если разрешить в драйвере режим мульти-PDO (подробнее см. п.3.4.1.4)

Сообщение PDO содержит в себе данные, набранные из данных других объектов словаря CANopen. Другими словами, отправляет значения выбранных пользователем объектов словаря в автоматическом режиме. Отображение объектов в PDO описывается структурой, называемой PDO-отображением (PDO-mapping). Это отображение связывает принимаемые/отправляемые данные с конкретными объектами словаря. Сам PDO-mapping также представлен в объектном словаре устройства, поэтому он может быть настроен мастером на подчинённых устройствах с помощью SDO-сервиса на стадии инициализации, либо самим устройством посредством пользовательской программы. Помимо настройки отображения каждое PDO имеет настройку, характеризующую его коммуникационные возможности, в частности идентификатор PDO-сообщения (COB-ID). Индексы (адреса) этих настроек в словаре объектов соответствуют следующим правилам:

- Индекс коммуникационной настройки RPDO = 1400h + № RPDO – 1;
- Индекс настройки отображения RPDO = 1600h + № RPDO – 1;
- Индекс коммуникационной настройки TPDO = 1800h + № TPDO – 1;
- Индекс настройки отображения TPDO = 1A00h + № TPDO – 1.

3.4.1. Настройка RPDO

3.4.1.1. Коммуникационная настройка RPDOx (x = 1…8)

Коммуникационная настройка RPDO включает в себя параметры:

Адрес*	Имя переменной	Описание
140#.01	co_RPDOxCOBID	Идентификатор принимаемого сообщения TPDO: 0x0180 + nodeID – для приема TPDO1 0x0280 + nodeID – для приема TPDO2 0x0380 + nodeID – для приема TPDO3 0x0480 + nodeID – для приема TPDO4 0x0200 + nodeID – для приема TPDO5 0x0300 + nodeID – для приема TPDO6 0x0400 + nodeID – для приема TPDO7 0x0500 + nodeID – для приема TPDO8, где nodeID – номер узла, от которого будем принимать PDO. Например, для приёма TPDO2 от узла №7: COB-ID = 0x0280 + 7 = 0x0287
140#.02	co_transType	Не используется.

* Примечание: # = x–1 – смещение индекса относительно базового адреса 1400h.

3.4.1.2. Настройка отображения RPDOx (x = 1…8)

Включает в себя параметры распаковки принятого PDO-сообщения:

<table>
<tr><th>Адрес*</th><th>Имя переменной</th><th>Описание</th></tr>
<tr><td>160#.01</td><td>co_RPDOx_1Mapping</td><td>Набор битовых полей:
<table>
<tr><td>31 16</td><td>15</td><td>7 0</td></tr>
<tr><td>Index</td><td>Subinde</td><td>Length</td></tr>
</table>
<table>
<tr><th>Биты</th><th>Поле</th><th>Описание</th></tr>
<tr><td>31-16</td><td>Index</td><td>«Индекс» объекта, в который будет распаковываться первый сегмент PDO-сообщения.</td></tr>
<tr><td>15-8</td><td>Subindex</td><td>«Подындекс» объекта, в который будет распаковываться первый сегмент PDO-сообщения.</td></tr>
<tr><td>7-0</td><td>Length</td><td>«Длина»: длина первого сегмента PDO-сообщения в битах – рассчитывается драйвером автоматически* по формату объекта.</td></tr>
</table>
* Примечание: если требуется проигнорировать N битов данных из PDO-сообщения, то значение N (от 1 до 63) следует поместить в поле «Index» (остальные поля должны быть равны нулю).</td></tr>
<tr><td>160#.02</td><td>co_RPDOx_2Mapping</td><td>Аналогичный набор битовых полей, описывающий куда будет помещен второй сегмент PDO-сообщения.</td></tr>
<tr><td>160#.03</td><td>co_RPDOx_3Mapping</td><td>Аналогичный набор битовых полей, описывающий куда будет помещен третий сегмент PDO-сообщения.</td></tr>
<tr><td>160#.04</td><td>co_RPDOx_4Mapping</td><td>Аналогичный набор битовых полей, описывающий куда будет помещен четвертый сегмент PDO-сообщения.</td></tr>
</table>

* Примечание: # = x–1 – смещение индекса относительно базового адреса 1600h.

Таким образом, для каждого PDO-сообщения возможно сконфигурировать отображение не более чем четырёх элементов пакета данных.

Для оперативного разрешения/запрещения приёма уже настроенного RPDO в программе пользователя можно использовать битовую маску «co_PDOReceiveEna». Установленный в 1 бит (0 – 7) разрешает приём соответствующего RPDOx (x = 1…8), например, для приёма RPDO1 и RPDO3:

```
co_vars1.co_PDOReceiveEna = 0x05; // 00000101b
```

По умолчанию co_PDOReceiveEna = 0xFF, то есть прием всех 8 RPDO разрешен.

3.4.1.3. Настройка вызова callback функций

В драйвере существует возможность по приему PDO вызывать callback функцию. В коде функции пользовательская программа может производить дополнительные действия.

Внимание

Вызов callback функции осуществляется сразу после распаковки принятого PDO (в соответствии с настроенным отображением PDO) до выхода из прерывания CAN по приему сообщения.

Типовыми применениями callback функций по приему RPDO являются:

- Контроль интервала приема RPDO – для многих технологических процессов реального времени важно, чтобы обратные связи поступали регулярно. Для контроля периода следования RPDO в пользовательской программе организуется счетчик, инкрементирующийся в пользовательской программе с заданным интервалом (например, 1мс). В callback функции данный параметр обнуляется (соотвтетствующий код должен написать пользователь в теле callback функции). Таким образом, пользовательская программа может контролировать значение данного счетчика. Пример: *если счетчик превысил значение 1000, то это означает, что от момента приема последнего RPDO прошло 1000мс*. По превышению счетчиком заданного таймаута пользовательская программа может предпринять соответствующие действия.
- Распаковка RPDO при работе драйвера в режиме мульти-PDO (описание режима см. в п.п. 3.4.1.4). В режиме мульти-PDO в один PDO объект принимаются сообщения одного типа от всех устройств сети (например, PDO1 от узлов от 1 до 127). В таком режиме отображение PDO настраивается на временные объекты. А уже в callback функции пользовательская программа осуществляет распаковку данных из временных объектов в нужные переменные - в callback функцию в качестве аргумента передается номер узла, от которого принято PDO, поэтому пользователь может осуществлять разную распаковку принятого PDO для каждого номера узла.

Для разрешения вызова callback функций по приему PDO существует параметр «settings.RX_PDO_Callback_ena» (входит в структуру «SCo_OdVars»). Если параметр «settings.RX_PDO_Callback_ena» равен 1, то при каждом приёме PDOx драйвер CANopen будет вызывать соответствующую callback-функцию «co_RPDOx_Callback» – см. п.4.2 «Конфигурация».

Параметр «settings.RX_PDO_Callback_ena» можно изменять в любой момент времени.

3.4.1.4. Режим мульти PDO

Если параметр «settings.MultiPDO_ena» (входит в структуру «SCo_OdVars») равен 1, то включается так называемый режим «Мульти-PDO» – возможность принимать в одно RPDOx сообщение TPDO от всех устройств в сети (а не только от одного, как это делается стандартно). Для каждого RPDOx становятся доступны 8 диапазонов принимаемых идентификаторов COB-ID:

0x0181…0x01FF – для приема TPDO1
0x0281…0x02FF – для приема TPDO2

0x0381…0x03FF – для приема TPDO3
0x0481…0x04FF – для приема TPDO4
0x0201…0x027F – для приема TPDO5
0x0301…0x037F – для приема TPDO6
0x0401…0x047F – для приема TPDO7
0x0501…0x057F – для приема TPDO8

Внимание

Для настройки RPDOx на приём требуемого диапазона идентификаторов необходимо присвоить соответствующей переменной «co_RPDOxCOBID» **любой идентификатор из этого диапазона**.

Двум RPDO одного устройства не может быть сопоставлен один и тот же диапазон идентификаторов – в этом случае поведение непредсказуемо.

В режиме мульти PDO распаковка принятых PDO производится также - в соответствии с настройкой отображения PDO. Соответственно, PDO сообщения одного диапазона идентификаторов будут распаковываться одинаково. Это неудобно, так как PDO сообщения одного типа, но от разных номеров узлов могут иметь совершенно разную структуру передаваемых данных. Поэтому обычно в режиме мульти PDO отображение PDO настраивается на временные объекты. А уже в callback функции пользовательская программа осуществляет распаковку данных из временных объектов в нужные переменные - в callback функцию в качестве аргумента передается номер узла, от которого принято PDO, поэтому пользователь может осущетсвлять разную распаковку принятого PDO для каждого номера узла.

Если флаг «settings.MultiPDO_ena» устанавливается после инициализации драйвера CANopen, то необходимо переинициализировать сетевую часть драйвера без восстановления параметров из энергонезависимой памяти, вызвав функцию «co_ReInit»:

```
co1_Init(&co1_vars); // инициализация драйвера
co1_vars.settings.MultiPDO_ena = 1;
co_ReInit(&co1_vars); // возвращает 1 в случае успеха
```

3.4.2. Настройка TPDO

3.4.2.1. Коммуникационная настройка TPDOx (x = 1…8)

Включает в себя параметры:

Адрес*	Имя переменной	Описание
180#.01	co_TPDOxCOBID	Идентификатор PDO-сообщения, отправляемого по каналу TPDOx. Данный идентификатор недоступен для редактирования, т.к. **формируется драйвером CANopen автоматически** по номеру узла nodeID данного устройства: 0x0180 + nodeID – для TPDO1 0x0280 + nodeID – для TPDO2 0x0380 + nodeID – для TPDO3 0x0480 + nodeID – для TPDO4 0x0200 + nodeID – для TPDO5 0x0300 + nodeID – для TPDO6 0x0400 + nodeID – для TPDO7 0x0500 + nodeID – для TPDO8. Например, для TPDO2, отправляемого узлом №7: COB-ID = 0x0280 + 7 = 0x0287
180#.02	co_transType	Не используется.
180#.03	co_TPDOxITime	«Inhibit Time» – минимально допустимый интервал отправки PDO-сообщения по каналу TPDOx. Т.е. сообщение

		будет отправлено не ранее, чем через это время после предыдущего. Задаётся в условных единицах, 1 ед. соответствует 100 мкс. Данный параметр ограничивает максимальную частоту отправки PDO-сообщений в случае, например, когда отправка слишком часто инициируется пользователем по какому-то событию.
180#.04	co_compatEntry	Не используется.
180#.05	co_TPDOxEventTimer	«Event Timer» – максимально допустимый интервал отправки PDO-сообщения по каналу TPDOx. Т.е. сообщение будет отправлено не позднее, чем через это время после предыдущего. Задаётся в миллисекундах. Данный параметр фактически определяет частоту автоматической отправки PDO-сообщений. Если значение параметра равно нулю, то автоматическая отправка не осуществляется.

* Примечание: # = x–1 – смещение индекса относительно базового адреса 1800h.

3.4.2.2. Настройка отображения TPDOx (x = 1…8)

Включает в себя параметры упаковки отправляемого PDO-сообщения:

<table>
<tr><th>Адрес*</th><th>Имя переменной</th><th>Описание</th></tr>
<tr><td>1A0#.01</td><td>co_TPDOx_1Mapping</td><td>Набор битовых полей:
<table>
<tr><td>31 16</td><td>15 8</td><td>7 0</td></tr>
<tr><td>Index</td><td>Subindex</td><td>Length</td></tr>
</table>
<table>
<tr><th>Биты</th><th>Поле</th><th>Описание</th></tr>
<tr><td>31-16</td><td>Index</td><td>«Индекс» объекта, значение которого будет помещено в первые N1 битов данных отправляемого PDO-сообщения.</td></tr>
<tr><td>15-8</td><td>Sub ndex</td><td>«Подындекс» объекта, значение которого будет помещено в первые N1 битов данных отправляемого PDO-сообщения.</td></tr>
<tr><td>7-0</td><td>Length</td><td>«Длина»: число N1 первых разрядов в PDO-сообщении – рассчитывается драйвером автоматически* по формату обекта.</td></tr>
</table>
* Примечание: если требуется пропустить N битов данных в отправляемом PDO-сообщении, то значение N (от 1 до 63) следует поместить в поле «Индекс» (остальные поля должны быть равны нулю).</td></tr>
<tr><td>1A0#.02</td><td>co_TPDOx_2Mapping</td><td>Аналогичный набор битовых полей, описывающий, из какого объекта будут взяты данные для следующих N2 битов PDO-сообщения.</td></tr>
<tr><td>1A0#.03</td><td>co_TPDOx_3Mapping</td><td>Аналогичный набор битовых полей, описывающий, из какого объекта будут взяты данные для следующих N3 битов PDO-сообщения.</td></tr>
<tr><td>1A0#.04</td><td>co_TPDOx_4Mapping</td><td>Аналогичный набор битовых полей, описывающий, из какого объекта будут взяты данные для следующих N4 битов PDO-сообщения.</td></tr>
</table>

* Примечание: # = x–1 – смещение индекса относительно базового адреса 1A00h.

Таким образом, для каждого PDO-сообщения возможно сконфигурировать отображение не более чем четырёх элементов пакета данных.

Для оперативного разрешения/запрещения отправки уже настроенного TPDO можно использовать в пользовательской программе битовую маску «co_PDOTransmitEna». Установленный в 1 бит (0 – 7) разрешает отправку соответствующего TPDOx (x = 1…8), например, для отправки TPDO4 и TPDO7:

```
co_vars1.co_PDOTransmitEna = 0x48; // 01001000b
```

По умолчанию co_PDOTransmitEna = 0xFF, то есть передача всех 8 TPDO разрешена.

Для принудительной отправки PDO (например, при каком-то событии) пользователь может воспользоваться битовыми флагами «co_PDOTransmitFlags». Установленный в 1 бит (0 – 7) инициирует отправку соответствующего TPDOx (x = 1…8).

Рассмотрим пример настройки PDO. Допустим, узлу №7 требуется с частотой 1 Гц отправлять по каналу TPDO2 параметры 301A.02 (типа int16), 41B8.05 (типа Uint32) и 5877.01 (двухбитовое поле) в в виде следующего пакета данных:

Биты	63 56	55 54	53 22	21 16	15 0
Данные	–	5877.01	41B8.05	–	301A.02

А узел №28 должен принимать эти данные по каналу RPDO4 и распаковать их в объекты 341E.01 (типа Uint32), 4F2B.00 (типа int16) и 5100.06 (двухбитовое поле).

Соответствующая конфигурация PDO будет выглядеть так:

Отправитель (producer) – узел №7

Словарь объектов

Адрес	Переменная	Значение	Примечание
1801.01	co_TPDO2COBID	0x0287	Только для чтения, вычисляется автоматически
1801.03	co_TPDO2ITime	1000	1000 → 100 мс (должно быть меньше 1000 мс)
1801.05	co_TPDO2EventTimer	1000	1000 мс → 1 Гц
1A01.01	co_TPDO2_1Mapping	0x301A0200	Начинаем упаковку пакета с объекта 301A.02
1A01.02	co_TPDO2_2Mapping	0x00060000	Пропускаем 6 битов
1A01.03	co_TPDO2_3Mapping	0x41B80500	Начиная с бита 22 пакуем объект 41B8.05
1A01.04	co_TPDO2_4Mapping	0x58770100	Начиная с бита 54 пакуем объект 5877.01
301A.02	gain	-1976	-1976 = 0xF848
41B8.05	speed	0x2216A031	
5877.01	error	0x3	2 бита

TPDO2

PDO-сообщение (COB-ID = 0x0287)

Биты	63 56	55 54	53 22	21 16	15 0
Длина	8	2	32	6	16
Данные	00...00	03	22 16 A0 31	00...00	F8 48

RPDO4

Приёмник (consumer) – узел №28

Словарь объектов

Адрес	Переменная	Значение	Примечание
1403.01	co_RPDO4COBID	0x0287	Принимаем TPDO2 от узла №7
1603.01	co_RPDO4_1Mapping	0x4F2B0000	Начинаем распаковку в объект 4F2B.00
1603.02	co_RPDO4_2Mapping	0x00060000	Пропускаем 6 битов
1603.03	co_RPDO4_3Mapping	0x341E0100	Продолжаем распаковку с бита 22 в объект 341E.01
1603.04	co_RPDO4_4Mapping	0x51000600	Продолжаем распаковку с бита 54 в объект 5100.06
341E.01	ext_speed	0x2216A031	
4F2B.00	ext_gain	-1976	-1976 = 0xF848
5100.06	ext_error	0x3	2 бита

Настройка PDO может осуществляться следующими способами:

- Модификация соответствующих объектов словаря устройства с помощью SDO-запросов со стороны мастера сети, либо, например, с помощью приложения UniCON.

- Установка соответствующих значений по умолчанию при редактировании словаря объектов (с помощью приложения COODEdit).
- Установка значений непосредственно в коде устройства. Это можно сделать либо один раз при инициализации (после вызова функции инициализации драйвера CANopen!), либо в любой другой момент времени (в фоновой программе), после чего необходимо переинициализировать сетевую часть CANopen без восстановления параметров из ЭнОЗУ, вызвав функцию «co_ReInit»:

```
co1_Init(&co1_vars); // инициализация драйвера
co1_vars. co_TPDO2_1Mapping = 0x301A0200;
co_ReInit(&co1_vars); // возвращает 1 в случае успеха
```

4. Реализация драйвера CANopen

4.1. Состав драйвера

CANOpen_drv.h	Основной заголовочный файл с прототипами функций драйвера.
co_Odvars.h	Заголовочный файл с прототипами служебных структур данных.
CANOpen_drv.c	Исходный код драйвера. Файл отсутствует, если драйвер поставляется в виде скомпилированной библиотеки.
co_Odvars.c	Исходный код с определениями/инициализацией служебных структур данных. Файл отсутствует, если драйвер поставляется в виде скомпилированной библиотеки.
libCANOpen_drv.a	Скомпилированная библиотека драйвера. Файл отсутствует, если драйвер поставляется в виде исходных кодов.
CANOpenUDfuncs.c	В этот файл вынесены пользовательские функции, «привязанные» к конкретному аппаратному обеспечению устройства и необходимые для работы драйвера (инициализация ножек процессора, работа с внешней энергонезависимой памятью, callback-функции и т.п.).
cood1.h	Заголовочный файл словаря объектов CANopen для модуля CAN0 процессора (исторически в драйвере он идёт под номером 1 – далее будет использоваться эта нумерация). Файл может отсутствовать, если модуль CAN1 не задействован.
cood2.h	Заголовочный файл словаря объектов CANopen для модуля CAN1 процессора (исторически в драйвере он идёт под номером 2 – далее будет использоваться эта нумерация). Файл может отсутствовать, если модуль CAN2 не задействован.
cood1.c	Словарь объектов CANopen для модуля CAN1 в виде проинициализированных таблиц-массивов. Данный файл создаётся с помощью приложения COODEdit. Файл может отсутствовать, если модуль CAN1 не задействован.
cood2.c	Словарь объектов CANopen для модуля CAN2 в виде проинициализированных таблиц-массивов. Данный файл создаётся с помощью приложения COODEdit. Файл может отсутствовать, если модуль CAN2 не задействован.
CANBlockTransferInterface.h	Заголовочный файл с прототипами функций интерфейса блочной передачи данных.
CANBlockTransferInterface.c	Исходный код функций интерфейса блочной передачи данных.

4.2. Конфигурация

Первоначальная конфигурация драйвера CANopen определяет, какие периферийные модули CAN процессора будут задействованы в проекте. Это осуществляется на этапе компиляции драйвера с помощью макроопределений в заголовочном файле «CANOpen_drv.h»:

```
#define CAN_1_ENABLE // разрешить работу с модулем CAN1
#define CAN_2_ENABLE // разрешить работу с модулем CAN2
```

Если закомментировать ненужное определение, то соответствующий этому модулю CAN код в драйвере также будет закомментирован, что уменьшит размер программы.

Очевидно, если драйвер поставляется в виде скомпилированной библиотеки, то включение/выключение этих макроопределений уже не влияет на код библиотеки. Однако, следует иметь в виду, что макроопределения влияют на код файлов CANOpenUDfuncs.c и CANBlockTransferInterface.c. Поэтому рекомендуется данные макроопределения настраивать соответственно используемой библиотеке. Если вы используете библиотеку, в которой разрешены оба модуля CAN, то и в заголовочном файле разрешите оба макроопределения.

Для работы с каждым модулем CAN определена своя структурная переменная типа «struct SCo_OdVars» (для неё определён тип «TCo_OdVars»). Для модуля CAN1 это переменная «co1_vars», для модуля CAN2 – «co2_vars». При вызове какой-либо функции драйвера ей передаётся указатель на нужную переменную (в виде адреса «&co1_vars»).

Далее, если это не оговорено особо, в качестве имени переменной указывается только название поля в структуре «struct SCo_OdVars», например: «co_protectBit». Полное имя для доступа к соответствующей переменной зависит от того, с каким модулем CAN мы работаем. Для модуля CAN1 полное имя будет «co1_vars.co_protectBit», для модуля CAN2 – «co2_vars.co_protectBit».

Часть функций, необходимых для работы драйвера, вынесена в отдельный файл «CANOpenUDfuncs.c», предназначенный для редактирования пользователем. В этом файле пользователь должен реализовать специфические функции, завязанные на внешнее аппаратное окружение процессора своего устройства:

```
void co_CAN1GpioInit()
```
Описание:
Функция настройки нужных портов ввода/вывода микроконтроллера на функцию сигнальных линий CAN_TX/CAN_RX модуля CAN1.
Вызывается драйвером CANopen при инициализации.

```
void co_CAN2GpioInit()
```
Описание:
Функция настройки нужных портов ввода/вывода микроконтроллера на функцию сигнальных линий CAN_TX/CAN_RX модуля CAN2.
Вызывается драйвером CANopen при инициализации.

```
void co_UserMemoryRead(const T_UserMemoryContext *p)
```
Описание:
Функция побайтового чтения из энергонезависимой памяти (ЭНОЗУ). Это может быть как пользовательская флэш-память на борту процессора, так и внешне подключаемая, например, по SPI-интерфейсу.
Вызывается драйвером CANopen при восстановлении значений параметров словаря объектов.
Параметры:
- p – указатель на структуру «T_UserMemoryContext» (объявлена в файле «co_ODvars.h»), через которую в функцию передаётся:
  - MemStartAddr – адрес (смещение в байтах) данных в ЭНОЗУ;
  - MCUStartAddr – указатель на массив в ОЗУ процессора;
  - data_length – размер данных в байтах.

```
void co_UserMemoryWrite(const T_UserMemoryContext *p)
```
Описание:
Функция побайтовой записи в ЭНОЗУ. Это может быть как пользовательская флэш-память на борту процессора, так и внешне подключаемая, например, по SPI-интерфейсу.
Вызывается драйвером CANopen при сохранении значений параметров словаря объектов.
Параметры:
- p – указатель на структуру «T_UserMemoryContext» (объявлена в файле «co_ODvars.h»), через которую в функцию передаётся:
  - MemStartAddr – адрес (смещение в байтах) данных в ЭНОЗУ;
  - MCUStartAddr – указатель на массив в ОЗУ процессора;
  - data_length – размер данных в байтах.

```
void co_CANToExtInterface_Send(TZCanMsg* MSG, Uint16 tag_CANnum)
```
Описание:
Функция, вызываемая драйвером CANOpen при необходимости отправки CAN сообщения во внешний интерфейс (например, UART).
Параметры:
- MSG – указатель на структуру «TZCanMsg» (объявлена в файле «co_ODvars.h»), через которую в функцию передаётся:
  - id – идентификатор сообщения;
  - data – 8-байтовый массив данных;
  - dlc – число реальных (полезных) байтов в массиве данных;
- tag_CANnum – номер модуля CAN (1/2).

```
void co_RPDOx_Callback(Uint16 nodeID, Uint16 tag_CANnum)
```
Описание:
Callback-функции, вызываемые драйвером CANopen при приёме PDO-сообщения по каналу RPDOx (x = 1…8). Вызов callback функции осуществляется сразу после распаковки принятого PDO (в соответствии с настроенным отображением PDO) до выхода из прерывания CAN по приему сообщения. Соответственно, время выполнения callback функции влияет на время занимаемое прерыванием CAN. Поэтому, чем меньше (быстрее) callback функция, тем лучше.
Параметры:
- nodeID – номер узла, от которого пришло сообщение;
- tag_CANnum – номер модуля CAN (1/2).

Функции вида (имя задаётся пользователем):
```
void func(Uint16 par, Uint16 tag_CANnum)
```
Описание:
Callback-функция, вызываемая драйвером CANopen при запросе к объекту словаря или его изменении. Привязка функции к объекту осуществляется при редактировании словаря с помощью приложения COODEdit на уровне индекса, и для корневого объекта функция вызывается при запросе к любому его подындексу.
Параметры:
- par – тип запроса:
  0 – чтение объекта(-ов) индекса через SDO сервис (вызов callback функции осуществляется сразу после обработки SDO запроса до выхода из прерывания CAN по приему сообщения);
  1 – запись объекта(-ов) индекса через SDO сервис (вызов callback функции осуществляется сразу после обработки SDO запроса до выхода из прерывания CAN по приему сообщения);
  2 – восстановление объекта(-ов) индекса значениями по умолчанию (заводскими уставками из памяти программ процессора) - вызов callback функции осуществляется из тела функции co_ODexpositor_instaurationDefault сразу после восстановления значения по умолчанию для соответствующего индекса;
  3 – восстановление объекта(-ов) индекса из ЭНОЗУ - вызов callback функции осуществляется из тела функции co_ODexpositor_paramInstauration сразу после восстановления значения из ЭнОЗУ для соответствующего индекса;
- tag_CANnum – номер модуля CAN (1/2).

4.3. Инициализация

Осуществляется вызовом функции «co1_Init(&co1_vars)» (для модуля CAN1) и/или «co2_Init(&co2_vars)» (для модуля CAN2):

```
Uint16 co1_Init(TCo_OdVars* ppc)
Uint16 co2_Init(TCo_OdVars* ppc)
```

Описание:

Функция инициализации драйвера CANopen для модуля CAN1/CAN2.

Вызывается пользователем в начале работы программы до фонового цикла (когда ещё запрещены прерывания).

Параметры вызова:

- ppc – указатель на структуру используемого модуля CAN драйвера CANopen. При вызове функции «co1_Init» это должен быть адрес «&co1_vars», функции «co2_Init» – адрес «&co2_vars».

Возвращаемое значение:

0 – в случае неудачи;

1 – в случае успеха.

Если это разрешено флагом «settings.LoadParamsFromUserMemory_ena = 1», то сначала функция инициализации производит загрузку значений редактируемых (всех типов доступа) параметров словаря объектов из энергонезависимой памяти устройства (ЭнОЗУ) в соответствующие переменные, используя пользовательскую функцию «co_UserMemoryRead» (см. п.4.2 «Конфигурация»). Очевидно, пользователь должен заранее (до вызова функции «coX_Init») проинициализировать драйвер работы с ЭнОЗУ. Затем функция «coX_Init» осуществляет инициализацию сетевой части драйвера, в частности, настраивает PDO-сервис.

Перед вызовом функции инициализации также необходимо сконфигурировать ещё ряд настроек, доступных через структуру «coX_vars.settings» (X – номер модуля CAN):

CAN_ISR_priority	Приоритет прерывания CAN. Можно использовать константы из файла «DSP.h»: #define IRQ_PRIORITY_TZ1 #define IRQ_PRIORITY_ADC 2 #define IRQ_PRIORITY_CAP 3 #define IRQ_PRIORITY_EQEP 3 #define IRQ_PRIORITY_10K 4 #define IRQ_PRIORITY_CAN 5 #define IRQ_PRIORITY_1K 6 Это значение передаётся в системную функцию ядра «NVIC_EncodePriority» – см. файл «core_cm4.h». При расстановке приоритетов прерываний следует учитывать, что прерывание CAN занимает около 1500 тактов (в пике может быть и более 10000 тактов - зависит от загрузки сети, типов сообщений сети, использования callback функций, пользовательского кода в callback функциях и т.п.)
MultiPDO_ena	Флаг разрешения приёма Мульти-PDO (см. п.3.4.1.4»).
RX_PDO_Callback_ena	Флаг разрешения вызова callback-функций при приёме PDO (см. п.3.4.1.3).
AutoBusON_ena	Флаг разрешения оставаться в работе даже при наличии большого количества ошибок на линии CAN. С одной стороны, это разрешение позволяет работать устройству в сильно зашумленной линии, но с другой – при отказе, например, сигнальной линии RX, устройство «забьёт» всю сеть, пытаясь отправить свою посылку.
BlockTransfer_ena	Флаг разрешения использования в драйвере функций блочной

	передачи (см. п.4.6 «Интерфейс блочной передачи»).

Пример кода инициализации:

```
co1_vars.settings.LoadParamsFromUserMemory_ena = 1;
co1_vars.settings.CAN_ISR_priority = IRQ_PRIORITY_CAN;
co1_vars.settings.MultiPDO_ena = 0;
co1_vars.settings.RX_PDO_Callback_ena = 0;
co1_vars.settings.AutoBusON_ena = 1;
co1_vars.settings.BlockTransfer_ena = 1;
co1_Init(&co1_vars);
```

```
Uint16 co_ReInit(TCo_OdVars* ppc)
```

Описание:

Функция переинициализации драйвера без восстановления параметров из ЭнОЗУ.

Данная функция вызывается в фоновой части программы и должна использоваться лишь для ПОВТОРНОЙ инициализации, в случах, если пользователь изменяет настройки сетевой части драйвера в своей программе (например, при переконфигурировании PDO).

Параметры вызова:

- ppc – указатель на структуру используемого модуля CAN драйвера CANopen.

Возвращаемое значение:

0 – в случае неудачи;

1 – в случае успеха.

Пример переинициализации после изменения настройки TPDO:

```
co1_Init(&co1_vars); // инициализация драйвера
co1_vars. co_TPDO2_1Mapping = 0x301A0200;
co_ReInit(&co1_vars);
```

4.4. Тактирование процессов

Процессы в подсистеме CANopen разделяются на быстрые и медленные. Быстрые процедуры, требующие строгой дискретизации по времени, выполняются в прерываниях:

- Периодическое прерывание с частотой 1 кГц:

  В пользовательский обработчик этого прерывания необходимо вставить вызов функции:

```
co_1ms_calc(&co1_vars); // для модуля CAN1
co_1ms_calc(&co2_vars); // для модуля CAN2
```

  Время выполнения функции co_1ms_calc составляет около 1000 тактов.

  Если прерывание 1кГц в пользовательском ПО отсутствует, его необходимо создать.

- Прерывание по приёму/передаче сообщений в сети CAN:

Данные прерывания и функции их обработки настраиваются внутри драйвера CANopen. От пользователя не требуется никаких действий. При каждом прерывании по приему посылки CAN обнуляется переменная «ISRDroppingCounter». Это можно использовать для контроля целостности сети, например, инкрементируя данную переменную в прерывании 1 кГц – если её значение превысит заданный таймаут, значит в сети отсутствует обмен данными.

Медленные процедуры, такие как сохранение/восстановление параметров из ЭНОЗУ, загрузка заводских настроек и т.п., выполняются в фоновом режиме. В фоновый цикл своей программы пользователь должен вставить вызов следующих функций:

```
co_background_calc(&co1_vars); // для модуля CAN1
co_background_calc(&co2_vars); // для модуля CAN2
```

Время выполнения функции co_background_calc не регламентировано и может составлять единицы секунд (например, при восстановлении значений объектов словаря из ЭнОЗУ), так как содержит в себе блокирующие вызовы функций работы с ЭнОЗУ, циклы ожидания и .т.п.

4.5. Функции работы с параметрами

4.5.1. Использование ЭнОЗУ (пользовательской памяти)

Драйвер CANopen обеспечивает сохранение параметров словаря объектов в энергонезависимую память (это может быть как встроенная «пользовательская память», так и внешняя eeprom, подключенная, например, через SPI, или I^2C).

Для хранения параметров драйвер занимает некоторый неразрывный диапазон адресов (для каждого модуля CAN свой диапазон). Начало области хранения параметров определяется пользователем с помощью двух макросов (см. файл co_ODvars.h):

```
//! Начальный адрес длЯ сохранениЯ в SPI памЯти словарЯ 1-го CAN
#define CO1_SPI_START_ADDR 0
//! Начальный адрес длЯ сохранениЯ в SPI памЯти словарЯ 2-го CAN
#define CO2_SPI_START_ADDR 3500
```

Следует иметь в виду, что адресация побайтовая.

Сохранение параметров в энергонезависимой памяти происходит по диапазонам и типам параметров в соответствии с таблицей:

Таблица 1 Карта сохранения объектов словаря в энергонезависимой памяти

Область параметров	Диапазон индексов	Тип параметров	
Область коммуникационных параметров	1000h…1FFFh	rw	обычные редактируемые параметры
	1000h…1FFFh	rwCRC	контрольная сумма
	1000h…1FFFh	rwp	редактируемые защищенные параметры
	1000h…1FFFh	rwpCRC	контрольная сумма
	1000h…1FFFh	rwps	редактируемые защищенные секретные параметры
	1000h…1FFFh	rwpsCRC	контрольная сумма
	2000h…2FFFh	rw	обычные редактируемые параметры
	2000h…2FFFh	rwCRC	контрольная сумма
	2000h…2FFFh	rwp	редактируемые защищенные параметры
	2000h…2FFFh	rwpCRC	контрольная сумма
	2000h…2FFFh	rwps	редактируемые защищенные секретные параметры
	2000h…2FFFh	rwpsCRC	контрольная сумма
Область пользовательских параметров	3000h…3FFFh	rw	обычные редактируемые параметры
	3000h…3FFFh	rwCRC	контрольная сумма
	3000h…3FFFh	rwp	редактируемые защищенные параметры
	3000h…3FFFh	rwpCRC	контрольная сумма
	3000h…3FFFh	rwps	редактируемые защищенные секретные параметры
	3000h…3FFFh	rwpsCRC	контрольная сумма
	4000h…4FFFh	rw	обычные редактируемые параметры
	4000h…4FFFh	rwCRC	контрольная сумма
	4000h…4FFFh	rwp	редактируемые защищенные параметры
	4000h…4FFFh	rwpCRC	контрольная сумма
	4000h…4FFFh	rwps	редактируемые защищенные секретные параметры
	4000h…4FFFh	rwpsCRC	контрольная сумма
	5000h…5FFFh	rw	обычные редактируемые параметры
	5000h…5FFFh	rwCRC	контрольная сумма
	5000h…5FFFh	rwp	редактируемые защищенные параметры
	5000h…5FFFh	rwpCRC	контрольная сумма
	5000h…5FFFh	rwps	редактируемые защищенные секретные параметры
	5000h…5FFFh	rwpsCRC	контрольная сумма

Объем занимаемой драйвером области памяти зависит от размера словаря объектов. Распределение адресов для каждого диапазона, каждого типа параметров осуществляется на этапе генерации файла словаря объектов «coodX.c» с помощью программы COODEdit.

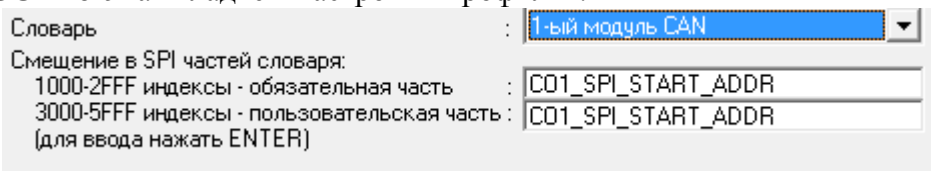
Так как энергонезависимой памятью может пользоваться не только драйвер CANOpen, но и другие программные модули – например, банк аварий, то программисту необходимо знать, какой объем пользовательской памяти занимает драйвер и где заканчивается область сохранения параметров в энергонезависимой памяти. Это можно сделать, посмотрев чему равен адрес хранения контрольной суммы последнего диапазона индексов (5000h…5FFFh), редактируемых защищенных секретных параметров. для этого в файле «coodX.c» необходимо найти строку с объявлением константы coX_SPIrange5rwpsCRC. Например, для CAN2 соответствующая строка может выглядеть так (файл cood2.c):

```
Uint16 const co2_SPIrange5rwpsCRC = CO2_SPI_START_ADDR+1164;
```

Таким образом, область память занимаемая драйвером CANopen для модуля CAN2 заканчивается на адресе (CO2_SPI_START_ADDR+1164+2), так как контрольная сумма занимает 2 байта. Аналогично можно вычислить конец области памяти модуля CAN1 – для этого необходимо в файле cood1.c найти константу co1_SPIrange5rwpsCRC.

4.5.2. Варианты распределения пользовательской памяти при использовании двух модулей CAN

При использовании в проекте двух модулей CAN с драйвером CANopen возможно несколько вариантов использования энергонезависимой памяти для хранения параметров драйвера. Выбор конкретного варианта определяется пользователем и формируется с помощью программы COODEdit на вкладке «Настройки профиля»:



Суть различных вариантов заключается в следующем. Как видно из рисунка, для диапазона обязательных (коммуникационных) индексов драйвера (1000h…2FFFh) и для диапазона пользовательских индексов 3000h…5FFFh имеется возможность задать индивидуальное смещение. Различные варианты представлены в таблице:

Таблица 2 Варианты распределения ЭнОЗУ при использовании обоих модулей CAN с драйвером CANopen

	Вариант 1		Вариант 2		Вариант 3		Вариант 4	
	CAN1	CAN2	CAN1	CAN2	CAN1	CAN2	CAN1	CAN2
Смещение коммуникационных параметров	1*	2**	1	2	1		1	
Смещение пользовательских параметров	1	2	1		1	2	1	

* - цифра 1 соответствует смещению CO1_SPI_START_ADDR;
** - цифра 2 соответствует смещению CO2_SPI_START_ADDR.

- **Вариант 1**. Каждый CAN занимает собственный диапазон адресов в ЭнОЗУ. Соответственно данный вариант наиболее затратный с точки зрения использования энергонезависимой памяти.

  Достоинства:

  Разделение коммуникационных областей позволяет получить два полностью независимых CANOpen узла с разными номерами узлов, скоростью передачи, настройками PDO, Heartbeat и т.д.

  Разделение пользовательских параметров позволяет иметь разные и независимые пользовательские параметры.

  Недостатки:

  Разделение пользовательских параметров накладывает некоторые ограничения на состав словарей. Так, например, пользователь может захотеть в оба словаря объектов

добавить индекс, ссылающийся на одну и ту же переменную в контроллере. Так делать можно, однако, если эта переменная будет доступна для записи (rw, rwp, или rwps параметр) из обоих драйверов, то пользователя может ожидать несколько подводных камней. Так как переменная отражена в оба CAN и в обоих CAN является переменной для записи, то очевидно, ее значение будет сохраняться в два разных места в ЭнОЗУ при сохранении параметров в каждом из драйверов. Соответственно, при инициализации контроллера последовательно будут инициализированы оба драйвера CANopen и последовательно осуществлена загрузка параметров из ЭнОЗУ сначала из одного драйвера, а потом из другого. Таким образом, пользователь может быть удивлен, когда обнаружит, что сохраненное в первом драйвере значение параметра не восстанавливается при инициализации контроллера. Тем не менее, данный вариант может быть использован пользователем на свое усмотрение. Для того, чтобы избежать проблем при восстановлении параметров пользователь может воспользоваться следующими рекомендациями при использовании данного варианта:

1. Из разных словарей стараться не ссылаться на одни и те же переменные контроллера (делать отдельные переменные для каждого драйвера).
2. Если пункт 1 по тем или иным причинам не соблюден, то есть объекты разных словарей ссылаются на одну переменную в контроллере, то сделать этот объект в одном из словарей только для чтения.
3. Если пункты 1 и 2 не соблюдены, то есть в обоих словарях имеются объекты для записи, ссылающиеся на одну и ту же переменную в контроллере, то отключить восстановление параметров из ЭнОЗУ при инициализации контроллера (для этого перед инициализацией драйвера вписать строку вида: coX_vars.settings.LoadParamsFromUserMemory_ena = 0) – это отключит восстановление значений параметров из ЭнОЗУ при инициализации и позволит избежать конфликта восстановления параметров. В этом случае восстановление параметров осуществляется пользователем управляемо в ручном режиме (из программы UniCON) или из пользовательской программы с помощью функции co_ODexpositor_paramInstauration).

- **Вариант 2**. Коммуникационные диапазоны индексов драйверов разделены, пользовательский диапазон индексов объединен. Данный вариант наиболее распространенный и **рекомендуемый к применению**.

  Достоинства:

  Разделение коммуникационных областей позволяет получить два полностью независимых CANopen узла с разными номерами узлов, скоростью передачи, настройками PDO, Heartbeat и т.д.

  Объединение пользовательских областей позволяет исключить возникновения конфликтов при восстановлении параметров и применять отображение одной переменной контроллера в оба драйвера CANopen.

  Недостатки:

  Объединение пользовательских областей предполагает, что пользовательская область индексов обоих словарей объектов драйверов одинакова.
- **Вариант 3 и Вариант 4**. Коммуникационные диапазоны индексов драйверов объединены. Данные варианты не применяются, так как для коммуникационных параметров драйвера имеют собственные независимые переменные, что предполагает разделение коммуникационных областей.

4.5.3. Функции сохранения и загрузки параметров

Драйвер CANopen предоставляет ряд сервисных функций для работы с параметрами объектов словаря:

```
void co_ODexpositor_instaurationDefault(
    TCo_OdVars* ppc, Uint16 type, Uint16 range)
```

Описание:

Функция восстановления значений параметров по умолчанию.
Данная функция должна вызываться в фоновой части программы.
Операция будет заблокирована (отменена), если:
type = 2 и co_protectBit = 1;
type = 3 и [co_protectBit = 1 или co_secretProtectBit = 1].

Параметры вызова:

- ppc – указатель на структуру используемого модуля CAN драйвера CANopen;
- type – тип параметров по уровню доступа (см. п.2.1 «Структура словаря»):
  1 – (обычные) редактируемые параметры;
  2 – редактируемые защищённые параметры;
  3 – секретные редактируемые параметры;
- range – затрагиваемый диапазон индексов объектов словаря:
  1 – 1000h…1FFFh;
  2 – 2000h…2FFFh;
  3 – 3000h…3FFFh;
  4 – 4000h…4FFFh;
  5 – 5000h…5FFFh.

```
void co_ODexpositor_paramConservation(
    TCo_OdVars* ppc, Uint16 type, Uint16 range)
```

Описание:

Функция сохранения значений параметров в ЭнОЗУ.
Данная функция должна вызываться в фоновой части программы.

Параметры вызова:

- ppc – указатель на структуру используемого модуля CAN драйвера CANopen;
- type – тип параметров по уровню доступа (см. п.2.1 «Структура словаря»);
- range – затрагиваемый диапазон индексов объектов словаря.

```
void co_ODexpositor_paramInstauration(
    TCo_OdVars* ppc, Uint16 type, Uint16 range)
```

Описание:

Функция восстановления значений параметров из ЭнОЗУ.
Данная функция должна вызываться в фоновой части программы.
Операция будет заблокирована (отменена), если:
type = 2 и co_protectBit = 1;
type = 3 и [co_protectBit = 1 или co_secretProtectBit = 1].

Параметры вызова:

- ppc – указатель на структуру используемого модуля CAN драйвера CANopen;
- type – тип параметров по уровню доступа (см. п.2.1 «Структура словаря»);
- range – затрагиваемый диапазон индексов объектов словаря.

Пример восстановления значений по умолчанию для редактируемых защищённых параметров из диапазона 3000h…3FFFh (для модуля CAN1):

```
    co1_vars.co_protectBit = 0;
    co_ODexpositor_instaurationDefault(&co1_vars, 2, 3);
    co1_vars.co_protectBit = 1; // если необходимо
```

Пример восстановления значений из ЭнОЗУ для всех объектов словаря CANopen (для модуля CAN2):

```
co2_vars.co_protectBit = 0;
co_ODexpositor_paramInstauration(&co2_vars, 1, 1);
 co_ODexpositor_paramInstauration(&co2_vars, 2, 1);
 co_ODexpositor_paramInstauration(&co2_vars, 3, 1);
 co_ODexpositor_paramInstauration(&co2_vars, 1, 2);
 co_ODexpositor_paramInstauration(&co2_vars, 2, 2);
 co_ODexpositor_paramInstauration(&co2_vars, 3, 2);
 co_ODexpositor_paramInstauration(&co2_vars, 1, 3);
 co_ODexpositor_paramInstauration(&co2_vars, 2, 3);
 co_ODexpositor_paramInstauration(&co2_vars, 3, 3);
 co_ODexpositor_paramInstauration(&co2_vars, 1, 4);
 co_ODexpositor_paramInstauration(&co2_vars, 2, 4);
 co_ODexpositor_paramInstauration(&co2_vars, 3, 4);
 co_ODexpositor_paramInstauration(&co2_vars, 1, 5);
 co_ODexpositor_paramInstauration(&co2_vars, 2, 5);
 co_ODexpositor_paramInstauration(&co2_vars, 3, 5);
```

```
Uint16* co_getAddr(TCo_OdVars* ppc, Uint32 ind_subind)
```

Описание:

Функция получения адреса переменной по её индексу/подындексу в словаре объектов.
Данную функцию можно вызывать как из фоновой программы, так и из прерывания.

Параметры вызова:

- ppc – указатель на структуру используемого модуля CAN драйвера CANopen.

Возвращаемое значение:

Указатель на переменную (для 32-битовой переменной – на её старшую часть), либо 0, если искомый объект не найден.

4.6. Интерфейс блочной передачи

Блочная передача данной реализации не входит в стандарт CANopen и является собственным дополнением ООО «НПФ ВЕКТОР». Блочная передача используется в случаях, когда необходимо передать большой объем низкоприоритетной информации, например, приложением UniCON при загрузке осциллограмм (на вкладке «Выборка») из устройства, обновлении ПО микроконтроллера по сети CAN и т.п. При этом CAN-сообщения потоковой передачи данных имеют идентификатор вида 0x0780 + № узла (самый низкий приоритет). То есть, идентификаторы блочной передачи не пересекаются со стандартными идентификаторами CANopen и все идентификаторы стандарта CANopen приоритетнее идентификаторов блочной передачи. Это гарантирует работу всех сервисов стандарта CANopen даже в условиях передачи больших объемов информации через блочную передачу.

Блочная передача реализует возможность потокового приема и передачи данных. Максимальный размер блока 256 16-и разрядных слов (512 байт). Целостность данных в принятом блоке контролируется с помощью контрольной суммы.

Так как в один момент времени можно передавать/принимать блок информации только из/в одного источника, а потенциальных источников/приемников информации в программе пользователя может быть много, то для исключения конфликтов использования блочной передачи используется дискретный автомат интерфейса блочной передачи – см. файл «CANBlockTransferInterface.c».

Порядок подключения интерфейса блочной передачи к проекту пользователя:

1. В главном файле проекта (например, «main.c») для каждого используемого модуля CAN определяем глобальную переменную типа «TCanBTInterface»:

```
TCanBTInterface Can1BTInterface = T_CANBT_INTERFACE_DEFAULTS;
TCanBTInterface Can2BTInterface = T_CANBT_INTERFACE_DEFAULTS;
```

2. В главном заголовочном файле проекта (например, «main.h») подключаем заголовочный файл:

```
#include "CANBlockTransferInterface.h"
```

И объявляем прототип глобальной переменной типа «TCanBTInterface» для каждого используемого модуля CAN:

```
extern TCanBTInterface Can1BTInterface; // для модуля CAN1
extern TCanBTInterface Can2BTInterface; // для модуля CAN2
```

3. В прерывание таймера 1 кГц вставляем вызов функции «SMCanBTInterface» для каждого используемого модуля CAN. Это можно сделать через поле «calc» структуры «TCanBTInterface», в которое при инициализации уже прописан (через макрос T_CANBT_INTERFACE_DEFAULTS) адрес этой функции:

```
Can1BTInterface.calc(&Can1BTInterface, NT_TIMER2->VALUE, &co1_vars);
Can2BTInterface.calc(&Can2BTInterface, NT_TIMER2->VALUE, &co2_vars);
```

```
void SMCanBTInterface(
    volatile struct SCanBTInterface* p, Uint32 time, TCo_OdVars* co_ptr)
```

Описание:
Функция дискретного автомата (ДА) интерфейса блочной передачи.
Вызывается пользователем с частотой 1 кГц.
Параметры:

- p – указатель на структуру интерфейса блочной передачи.
- time – 32-битное значение счётчика таймера, тактируемого частотой процессора. Его значение не должно быть ограничено, т.е. должно изменяться в диапазоне 0…0xFFFFFFFF.
- co_ptr – указатель на структуру используемого модуля CAN драйвера CANopen.

Работа с интерфейсом блочной передачи предполагает редактирование пользователем функции «SMCanBTInterface» (в файле «CANBlockTransferInterface.c»). По умолчанию, в дискретном автомате (ДА) интерфейса блочной передачи реализована только функция отправки данных «быстрого» осциллографа по запросу приложения UniCON (соответствующий номер состояния ДА задан константой CANBT_INTERFACE_DATALOG).

Предлагается следующий порядок реализации пользовательской функции отправки:

- Определяем в файле «CANBlockTransferInterface.h» номер своего состояния для отправки данных, например:

```
#define CANBT_INTERFACE_MY_TX  3
```

- Проверяем, разрешена ли блочная передача, и свободен ли её интерфейс, после чего переводим ДА в нужное состояние через переменную «co_blockTransferCommand»:

```
if (co1_vars.settings.BlockTransfer_ena &&
    (co1_vars.co_blockTransferCommand == CANBT_INTERFACE_FREE))
{
    co1_vars.co_blockTransferCommand = CANBT_INTERFACE_MY_TX;
}
```

Эта процедура выполняется не в коде функции «SMCanBTInterface». Например, для отправки данных «быстрого» осциллографа это осуществляется в callback-функции «callback_dlog» (файл «CANOpenUDfuncs.c»), вызываемой при SDO-запросе к параметру 5151.00.

- Далее, в функции «SMCanBTInterface», реализуем обработчик своего состояния.
  При первом вхождении в состояние подготавливаем массив передаваемых данных (в принципе, это можно сделать заранее в другом месте, например, в фоновой программе), и вызываем функцию «CANBlockTransferInitTX», инициирующую передачу драйвером CANopen. После чего, при каждом вхождении в обработчик состояния, проверяем статус отправки:

```
case CANBT_INTERFACE_MY_TX:
{
    if (p->E) // флаг E – признак первого вхождения в состояние
    {
```

```
            int16 buff[256];
            ...
            заполняем массив buff своими данными
            ...
            CANBlockTransferInitTX(co_ptr, 256, (Uint16*)&buff);
        }
        if (co_ptr->BT->tx_status != CANBT_STATUS_BUSY) // отправлено?
        {
            // Освобождаем интерфейс
            co_ptr->co_blockTransferCommand = CANBT_INTERFACE_FREE;
        }
        break;
    }
```

```
void CANBlockTransferInitTX(
    TCo_OdVars* ppc, Uint16 size, Uint16* source_ptr)
```

Описание:

Функция инициирования передачи блочных данных.

Параметры вызова:

- ppc – указатель на структуру используемого модуля CAN драйвера CANopen.
- size – размер передаваемых данных в словах (1–256).
- source_ptr – указатель на массив передаваемых данных.

Возвращаемое значение:

Функция возвращает управление сразу после вызова, при этом устанавливается TX-статус:
ppc->BT->tx_status = CANBT_STATUS_BUSY, если параметры вызова корректны, либо
ppc->BT->tx_status = CANBT_STATUS_ANY_ERR, если задан неверный размер данных.
Функция также устанавливает RX-статус сервиса приёма в режим ожидания:
ppc->BT->rx_status = CANBT_STATUS_IDLE.
Далее TX-статус может принять следующие значения:
CANBT_STATUS_TIMEOUT – данные не удалось отправить за 500 мс после вызова функции
CANBT_STATUS_SUCCESSFUL – данные успешно отправлены

Приём данных осуществляется похожим образом. В нужном месте переводим ДА в состояние приёма (для примера, CANBT_INTERFACE_MY_RX):

```
    if (co1_vars.settings.BlockTransfer_ena &&
        (co1_vars.co_blockTransferCommand == CANBT_INTERFACE_FREE))
    {
        co1_vars.co_blockTransferCommand = CANBT_INTERFACE_MY_RX;
    }
```

Далее, в функции «SMCanBTInterface», реализуем обработчик своего состояния, предполагая, что у нас существует некий буфер «buff» для приёма данных:

```
    case CANBT_INTERFACE_MY_RX:
    {
        if (p->E) // флаг E – признак первого вхождения в состояние
        {
            CANBlockTransferInitRX(co_ptr, 256, (Uint16*)&buff);
        }
        if (co_ptr->BT->rx_status != CANBT_STATUS_BUSY) // получено?
        {
            // Данные приняты успешно?
            if (co_ptr->BT->rx_status == CANBT_STATUS_ SUCCESSFUL)
            {
                ... обрабатываем данные, принятые в буфер «buff»
            }
            // Освобождаем интерфейс
            co_ptr->co_blockTransferCommand = CANBT_INTERFACE_FREE;
```

```
        }
        break;
    }
```

```
void CANBlockTransferInitRX(
    TCo_OdVars* ppc, Uint16 size, Uint16* dest_ptr)
```
Описание:
Функция инициирования приёма блочных данных.
Параметры вызова:
- ppc – указатель на структуру используемого модуля CAN драйвера CANopen.
- size – ожидаемый размер данных в словах (1–256).
- dest_ptr – указатель на массив, куда будут помещены принятые данные.

Возвращаемое значение:
Функция возвращает управление сразу после вызова, при этом устанавливается RX-статус:
ppc->BT->rx_status = CANBT_STATUS_BUSY , если параметры вызова корректны, либо
ppc->BT->rx_status = CANBT_STATUS_ANY_ERR, если задан неверный размер данных.
Функция также устанавливает TX-статус сервиса отправки в режим ожидания:
ppc->BT->tx_status = CANBT_STATUS_IDLE.
Далее RX-статус может принять следующие значения:
CANBT_STATUS_TIMEOUT – за 500 мс после вызова функции не приняты никакие данные
CANBT_STATUS_CRC_ERR – принятые данные имеют неверную контрольную сумму
CANBT_STATUS_ANY_ERR – размер принятых данных не совпадает с ожидаемым («size»)
CANBT_STATUS_SUCCESSFUL – данные успешно приняты

4.7. Взаимодействие с драйвером CANopen через другие интерфейсы связи

Существует возможность обращения к объектам словаря драйвера CANOpen через другие интерфейсы связи, например UART. Это может быть полезным в случаях, когда пользователь не имеет возможности взаимодействовать с микроконтроллером через CAN, например, если CAN задействован для других протоколов связи (не CANopen), либо при отсутствии переходника интерфейсов USB-CAN и т.п.

В таком режиме взаимодействие с объектами словаря осуществляется **только** посредством сервиса SDO. Для этого драйвер предоставляет пользователю функцию:

```
void Z_co_receiveSDOrequest (TCo_OdVars* ppc, TZCanMsg *MSG)
```
Описание:
Функция обработки SDO запроса к драйверу (чтение /запись объекта словаря)
Параметры вызова:
- ppc – указатель на структуру используемого модуля CAN драйвера CANopen.
- `MSG` – указатель на структуру с данными CAN сообщения.

Возвращаемое значение:
-

Кроме того, пользователь должен в файле «CANOpenUDfuncs.c» написать тело функции «co_CANToExtInterface_Send», реализующей собственно отправку CAN сообщения через внешний интерфейс связи.

```
void co_CANToExtInterface_Send(TZCanMsg* MSG, Uint16 tag_CANnum)
```
Описание:
Функция, которая вызывается драйвером CANOpen при необходимости отправки CAN сообщения во внешний интерфейс. Внутрь функции пользователем должен быть вставлен соответствующий код, осуществляющей передачу CAN пакета.
Параметры вызова:
- `MSG` – указатель на структуру с данными CAN сообщения.
- `tag_CANnum` – номер модуля CAN (1/2).

Возвращаемое значение:
-

При этом реализация непосредственно транспорта CAN сообщений через внешний интерфейс, инициализация соответствующей аппаратной части осуществляется пользователем.

Пример, использования функций взаимодействия с драйвером CANopen через внешний интерфейс связи пользователь может найти в файле «v_CANtoRS.c», в котором реализуется взаимодействие программы UniCON с драйвером CANOpen микроконтроллера через интерфейс RS232.

5. Минимально необходимая конфигурация профиля CANopen

В следующей таблице описывается минимальная конфигурация объектов словаря, необходимая для корректного отображения параметров устройства в приложении UniCON:

<table>
<tr><th>Объект(ы)</th><th>Комментарий</th></tr>
<tr><td colspan="2">Общие коммуникационные параметры</td></tr>
<tr><td>1018h.02h</td><td>В младших 16-ти разрядах этого параметра должен содержаться код продукта в соответствии с номером текста из перечисления №56 программы COODEdit. Перечисление №56 является служебным и должно содержать список названий всех устройств производителя.
В старших 16-ти разрядах этого параметра опционально может кодироваться дополнительная информация об изделии, например, его модификация. Каждый ниббл (полубайт) этого значения кодирует один символ (т.е. всего – 4 символа) согласно следующей схеме:
▪ коды от 0 до 9 выводятся как есть
▪ A – символ ‘A’
▪ B – пробел
▪ C – запятая
▪ D – точка
▪ E – дефис
▪ F – слеш (дробь)
Для примера предположим, что перечисление №56 содержит следующие тексты:
0: Калькулятор
1: Преобразователь
2: Конвертор
Если параметр 1018h.01h содержит значение 6F0B 0001h, то приложение UniCON выведет название устройства как «Преобразователь-6/0 » (дефис между названием и дополнительной информацией ставится автоматически).
Данный параметр – только для чтения, соответствующая ему переменная должна явно инициализироваться в программном коде устройства:
<pre>co1_vars.co_productCode = 0x6F0B0001; // если используется CAN1
co2_vars.co_productCode = 0x6F0B0001; // если используется CAN2</pre></td></tr>
<tr><td>1018h.03h</td><td>В младших 16-ти разрядах этого параметра должен содержаться номер ревизии (версия) ПО. Каждый ниббл (полубайт) этого значения кодирует шестнадцатеричную цифру номера версии в формате HH.LL (HH – мажорный номер версии, LL – минорный). Например, значение 2345h будет выведено как v.23.45.
В старших 16-ти разрядах этого параметра опционально может кодироваться дата выпуска версии в виде битовых полей:
<table><tr><td>15 9</td><td>8 5</td><td>4 0</td></tr><tr><td>Year</td><td>Month</td><td>Day</td></tr></table>
<table><tr><th>Биты</th><th>Поле</th><th>Описание</th></tr><tr><td>15-9</td><td>Year</td><td>Год от 00 до 99</td></tr><tr><td>8-5</td><td>Month</td><td>Месяц от 1 до 12</td></tr><tr><td>4-0</td><td>Day</td><td>День от 1 до 31</td></tr></table>
Например, значение 1556h будет выведено как дата 22.10.2010.
Данный параметр – только для чтения, соответствующая ему переменная должна явно инициализироваться в программном коде устройства:
<pre>co1_vars.co_revisionNumber = 0x15562345; // если используется CAN1
co2_vars.co_revisionNumber = 0x15562345; // если используется CAN2</pre></td></tr>
<tr><td colspan="2">Дополнительные параметры связи</td></tr>
<tr><td>2001h.00h</td><td>Скорость обмена в сети CAN:</td></tr>
</table>

<table>
<tr><td></td><td>2 – 50 кбит/с
3 – 125 кбит/с (значение по умолчанию)
4 – 250 кбит/с
5 – 500 кбит/с
6 – 1 Мбит/с
Данный параметр является редактируемым, т.е. его значение хранится в энергонезависимой памяти устройства (если оно поддерживает эту функцию). При необходимости соответствующая переменная может быть явно проинициализирована в программном коде устройства:
<code>co1_vars.co_bitRate = 3; // если используется CAN1</code>
<code>co2_vars.co_bitRate = 3; // если используется CAN2</code></td></tr>
<tr><td>2002h.02h</td><td>Данный параметр является необязательным и дублирует параметр 1018h.02h «Код продукта» (см. выше). Он может быть добавлен в словарь, чтобы пользователь мог посмотреть это значение, т.к. по стандарту CANopen параметры с адресами ниже 2000h не должны быть доступны пользователю.</td></tr>
<tr><td>2002h.03h</td><td>Данный параметр является необязательным и дублирует параметр 1018h.03h «Номер и дата версии ПО» (см. выше). Он может быть добавлен в словарь, чтобы пользователь мог посмотреть это значение, т.к. по стандарту CANopen параметры с адресами ниже 2000h не должны быть доступны пользователю.</td></tr>
<tr><td colspan="2">Масштабирующие коэффициенты (МК)</td></tr>
<tr><td>2100h.01h
…
211Fh.01h</td><td>В данных параметрах задаются значения масштабирующих коэффициентов 00h – 1Fh.
Масштабирующие коэффициенты (МК) имеют размер 16 бит и служат для преобразования относительных величин в абсолютные (физические) путём их умножения на величину МК.
Например, если параметр «Ток» (точнее, соответствующая ему переменная в коде устройства) вычисляется в условных единицах (от 0 до 1) относительно номинального значения 50А, то для вывода его в амперах необходимо в программном коде устройства поместить в соответствующий МК значение номинального тока:
<code>co1_vars.co_scaleNum12 = 50; // если используется CAN1</code>
<code>co2_vars.co_scaleNum12 = 50; // если используется CAN2</code>
(здесь, для примера, предполагается, что параметр «Ток» привязан к МК №12h)
Масштабирующй коэффициент может быть не только целочисленным, но и дробным (с фиксированной точкой). Формат МК задаётся с помощью параметров 2100h.02h … 211Fh.02h (см. ниже).</td></tr>
<tr><td>2100h.02h
…
211Fh.02h</td><td>Здесь в виде битовых полей задаётся формат значений соответствующих масштабирующих коэффициентов 00h – 1Fh:
<table>
<tr><td>15 10</td><td>9 7</td><td>6 5</td><td>4 0</td></tr>
<tr><td>Units</td><td>Prefix</td><td>Precision</td><td>ScaleQ</td></tr>
</table>
<table>
<tr><th>Биты</th><th>Поле</th><th>Описание</th></tr>
<tr><td>15-10</td><td>Units</td><td>Идентификатор (номер) размерности из служебного перечисления №7. Например, номер 3 – это «Гц». Соответственно, параметр, связанный с данным масштабирующим коэффициентом (МК) при выводе на дисплей будет иметь размерность «#Гц», где # – префикс, определяемый следующим битовым полем:</td></tr>
<tr><td>9-7</td><td>Prefix</td><td>Идентификатор (номер) префикса размерности из служебного перечисления №6. Например, номер 1 – это префикс «к». Соответственно, параметр, связанный с данным масштабирующим коэффициентом при выводе на экран будет иметь размерность «кГц».</td></tr>
<tr><td>6-5</td><td>Precision</td><td>Количество знаков после запятой при выводе</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><td></td><td>
<table>
<tr><td></td><td></td><td>масштабируемой величины на экран.</td></tr>
<tr><td>4-0</td><td>ScaleQ</td><td>Количество дробных разрядов масштабирующего коэффициента. Например, если здесь указано число 6, то инициализация МК значением 50 будет выглядеть так:
<code>co1_vars.co_scaleNum12 = 50 << 6; // для CAN1</code>
<code>co2_vars.co_scaleNum12 = 50 << 6; // для CAN2</code></td></tr>
</table>
Например, значение 0x24C6 интерпретируется следующим образом:
размерность = 9 («Вт»), префикс = 1 («к»), число знаков после запятой = 2, число дробных разрядов МК = 6.
Данные параметры являются редактируемыми, их значения сохраняются в энергонезависимой памяти устройства (если оно поддерживает эту функцию).
При редактировании словаря объектов в приложении COODEdit (на вкладке «Масштабирующие коэффициенты») реализовано интерактивное редактирование форматов МК. Соответствующие установки автоматически прописываются в качестве значений по умолчанию для данных параметров.
При необходимости соответствующий формат может быть явно проинициализирован в программном коде устройства:
<code>co1_vars.co_scaleNum12Format = 0x24C6; // если используется CAN1</code>
<code>co2_vars.co_scaleNum12Format = 0x24C6; // если используется CAN2</code></td></tr>
<tr><td>2100h.03h
…
2100h.06h
……
211Fh.03h
…
211Fh.06h</td><td>Данные параметры привязаны к разным битовым полям той же переменной, что и параметры 2100h.02h … 211Fh.02h (co_scaleNumXXFormat). Это сделано исключительно для удобства редактирования форматов масштабирующих коэффициентов (МК). Если такой режим настройки форматов МК не требуется, эти битовые параметры можно удалить из словаря объектов.</td></tr>
<tr><td colspan="2">Служебные параметры</td></tr>
<tr><td>2502h</td><td>Слово управления CAN (необязательный параметр).
В данной реализации драйвера CANopen, что под этим индексом сгруппированы однобитные флаги для подачи различных команд устройству (Пуск/Стоп и т.п.). После выполнения команды ПО устройства обнуляет соответствующий бит переменной. Например, параметр 2502h.01h – команда «Пуск» – привязана к биту №0 переменной cmd.all. ПО устройства периодически анализирует состояние этого бита и после выполнения команды его обнуляет:
<pre>if ((cmd.all & 0x01) != 0)
{
    … выполнение команды …
    cmd.all &= (~0x01);
}</pre>При настройке профиля устройства в приложении UniCON к этим параметрам могут быть привязаны кнопки на панели управления для оперативной подачи команд. Если объект существует, программа UniCON в функции автоматического создания профиля устройства создает кнопки оперативного управления на панели управления на основе параметров (подындексов) в данном индексе.</td></tr>
<tr><td>2532h.01h</td><td>В проектах ООО «НПФ ВЕКТОР» принято, что этот параметр отображает номер (код) аварии в приложении UniCON или на пульте оперативного управления.
Для отображения аварии в текстовом виде название параметра (его текст) должно быть связано с соответствующим перечислением. Если объект существует, программа UniCON в функции автоматического создания профиля устройства создает отображение данной переменной на панели управления в качестве кода аварии.</td></tr>
<tr><td>2532h.02h</td><td>Расширение кода аварии (необязательный параметр).
Здесь может храниться дополнительный код, уточняющий, в какой подсистеме</td></tr>
</table>

	устройства произошла авария, например, номер силового ключа. Для отображения этой дополнительной информации в текстовом виде название параметра (его текст) должно быть связано с соответствующим перечислением.

2534h.01h	Для корректного отображения названий групп параметров в приложении UniCON в словаре объектов должен присутствовать корневой объект 2534h.00h, а внутри него – параметр 2534h.01h типа UINT16. В этом параметре должен храниться номер перечисления с названиями групп параметров.
	Данный параметр является редактируемым, т.е. его значение хранится в энергонезависимой памяти устройства (если оно поддерживает эту функцию). При необходимости соответствующая переменная может быть явно проинициализирована в программном коде устройства:
	`disp_group_number = 129;`
	Другой вариант – использовать в качестве номера перечисления переменную-константу, создаваемую при генерации файла словаря объектов на языке Си программой COODEdit для встраивания в ПО контроллера (cood.c):
	`Uint16 const co1_groupEnum = 129; // если используется CAN1`
	`Uint16 const co2_groupEnum = 129; // если используется CAN2`
	Эта константа присваивается равной номеру перечисления, выбранному в настройках профиля в поле выбора перечисления групп параметров в COODEdit.

Параметры и переменные пользовательского проекта	
3000h … 5FFFh	Здесь располагаются пользовательские параметры проекта ПО устройства.